

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-ac5b4d2ea0959274a.jsonl`  
- SHA-256: `e70eb5e15ba52b79331e69bd0c31701ecfac1fef0a9bae4371c4f9147d29a55`  
- Source modified: `2026-07-02T04:58:23+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_7049e1ed-b93`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T04:45:39.417Z)

You are a senior engineer producing ONE isolated PR. Today is 2026-07-02.

REPO: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (a shared checkout ? follow the safety rules exactly).

AUDIT FINDINGS YOU ARE FIXING: R2-4 (no ffmpeg timeouts, wedged single-worker queue) ? Grep C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md for these IDs and read the full evidence + verifier notes before coding.

TASK BRIEF:

In C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer: add timeout= to every SERVING-PATH ffmpeg/ffprobe subprocess.run call: backend/pipeline/stitcher/compiler.py:130 (ffprobe), :321 (per-clip encode), :494 (concat) and backend/pipeline/validators/format.py probes (grep for other subprocess ffmpeg call sites on the /api/process + /api/compile paths; SKIP offline research/training scripts). Timeouts must scale sanely: probes ~60s fixed; encodes proportional to clip duration (e.g. max(300, duration*10)) ? read the call sites and pick honest bounds; on TimeoutExpired kill the process tree and surface a clean job failure (result.status=failed with a distinct reason), never a crash. The in-code '02 RISK' comment at backend/api/job_worker.py:32-39 documents the wedge ? update it to note timeouts now bound the ffmpeg legs. Tests: mock subprocess to raise TimeoutExpired and assert the clean-failure path per call site touched.

BRANCHING SAFETY (mandatory ? shared checkout, sibling sessions exist):

1. In the MAIN repo checkout run: git fetch origin
2. Create an ISOLATED WORKTREE: git worktree add "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-<shortname>" -b <branch-name> origin/master (branch explicitly FROM origin/master ? never from the current local branch; use origin/main for repos whose default is main)
3. Do ALL work inside the worktree. Before committing: git branch --show-current + git log --oneline -1 must show YOUR branch on the origin-master base.
4. Stage EXPLICIT paths only (git add <file> <file>) ? NEVER git add -A / . / -u
5. Before opening the PR: git log --oneline origin/master..HEAD must list ONLY your commits.

COMMIT/PR CONVENTIONS:

- Commit message: conventional style matching the repo's history; end the message with a blank line then: Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>
- PR body: what/why, a "## Verification" section stating exactly what you ran locally and what CI covers, and end with: ? Generated with [Claude Code](https://claude.com/claude-code)
- Do NOT merge the PR. Do NOT remove the worktree (a fixer may need it). Return the PR number, branch, worktree path.

QUALITY BAR:

- Read the repo's CLAUDE.md first; for the engine also docs/CODE_MAP.md §0 (file placement) before adding any new file.
- Add/extend tests for every behavior change. Run the targeted tests you touched (python -m pytest tests/<relevant> -q) + ruff check on changed files; if mypy config exists, run mypy on touched modules. Full suite is CI's job ? do not run it.

- Default-safe: new knobs default OFF / behavior-preserving unless the task explicitly says otherwise.
- The full audit evidence is in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep it for your finding IDs and READ the verifier notes; they contain corrections (exact file:line, scoping traps) you MUST honor.

If the brief's premise turns out false against current code (something already fixed, file moved), adapt honestly and say so in notes ? never force a pointless diff. If you genuinely cannot complete (env broken, auth missing), open NO PR and explain in notes with pr_number=0.

2. user (2026-07-02T04:45:47.263Z)

444-- verdict: CONFIRMED
445-- reasoning: Every element of the claim reproduces in the current working tree (rally-annotator @ main 3d604bc, clean):
446-
447-(1) Runs on every page: web/entrypoints/content.ts:13 `matches: ["<all\_urls>"]` (top frame only per line 16), and web/wxt.config.ts:13 `host\_permissions: ["<all\_urls>"]`. content.ts:32-44 unconditionally calls `mountPanel(...)` then `panel.toggle()` ? the panel starts hidden (display:none) but stays mounted.
448-
449-(2) 3 Hz loop regardless of visibility: web/src/ui/panel.ts:376 `const clock = window.setInterval(render, 333);` starts at mount and is cleared only in `destroy()` (panel.ts:395), which is never called from production code ? only from web/test/panel.smoke.test.ts. `toggle()` (panel.ts:391-393) merely flips display:none; it does not pause the interval.
450-
451-[Omitted long context line]
452-
453-(4) No fix landed: git log for content.ts/panel.ts/directVideo.ts shows only ad6ac9c (initial), 5ec1831 and 0a05ecf (i18n) ? no gating change. No visibilitychange/IntersectionObserver/MutationObserver/activeTab anywhere in web/ (grep: only hit is the setInterval itself).
454-
455-(5) Intent: `<all\_urls>` is deliberate (DESIGN.md LD-7: panel hidden by default, icon is the explicit "annotate this page" gesture) ? but LD-7 gates only the UI, not the scan loop, so the perf cost contradicts the design's own "not intrusive" rationale.
456-[Omitted long context line]
457-
458-Severity P2 not P1: no correctness/data-loss/security impact, and the extension is version 0.1.0/private (not store-published), so blast radius today is dev machines; would be P1 if shipped broadly.
459-
460-[Omitted long context line]
461-
462-### R2-2 [P2/S] ? rally-annotator
463-**Locked decision LD-i8 (web language persisted + seeded from browser language) is documented as shipped but not implemented ? panel always boots in English and forgets the choice on reload**
464-- verdict: CONFIRMED
465-[Omitted long context line]
466-[Omitted long context line]
467-
468-### R2-3 [P2/S] ? sports-data-collector
469-**Collector pins obsreport as a plain PyPI requirement while the name is unclaimed on PyPI ? dependency-confusion window and install broken today**
470-- verdict: CONFIRMED
471-[Omitted long context line]
472-[Omitted long context line]
473-
474-### R2-4 [P2/S] ? badminton-highlight-indexer
475-**No timeout on any serving-path ffmpeg subprocess ? one hung ffmpeg permanently wedges the single-worker job queue**

```

476-- verdict: CONFIRMED
477-[Omitted long context line]
478-[Omitted long context line]
479-
480-### R2-5 [P1/S] ? badminton-highlight-indexer
481-**Engine CLAUDE.md 'Current state' section is ~3 weeks false ? claims platform/owned-model
work 'has not started'**
482-- verdict: CONFIRMED
483-[Omitted long context line]
484-[Omitted long context line]
485-
486-### R2-6 [P1/M] ? sports-data-collector
487-**Golden-corpus registry rot: all 6 owned 'curated' videos point at a deleted absolute
local_path and the 4.9GB source videos are single-copy with no documented backup**
488-- verdict: CONFIRMED
489-[Omitted long context line]
490-[Omitted long context line]
491-
492-### R2-7 [P2/S] ? non-git
493-**Roora vendor pilot record (paid deliverables + corrected golden labels + 3 videos) is a
single unversioned local copy at risk of loss**
494-- verdict: CONFIRMED
495-[Omitted long context line]
496-[Omitted long context line]
497-
498-### R2-8 [P1/S] ? badminton-highlight-indexer
499-**Upload retention sweep tool exists but nothing schedules it ? unbounded upload growth +
unenforced 7-day privacy retention on any hosted deployment (#301)**
500-- verdict: CONFIRMED
501-[Omitted long context line]
502-[Omitted long context line]
503-
504-

```

3. user (2026-07-02T04:45:50.670Z)

master...origin/master

4. user (2026-07-02T04:45:59.380Z)

```

- verdict: CONFIRMED
- reasoning: (1) Bare PyPI-style pin exists in current tree: C:/Users/avidu/Projects/khelsutra-
guru/sports-data-collector/pyproject.toml:38-40 ? `report = ["obsreport>=0.1.3"]`, with the
comment at lines 32-37 itself admitting the intended form is a git+ssh direct reference. (2)
Name is unclaimed on PyPI today (2026-07-02): https://pypi.org/pypi/obsreport/json returns 404,
and `pip index versions obsreport` returns "ERROR: No matching distribution found for obsreport"
(personally observed). (3) The private package really is named `obsreport` (sports-
obsreport/pyproject.toml:6, version 0.1.3, tag v0.1.3 exists) and lives at the private repo
Khelsutra/sports-obsreport (gh api confirms private=true; avidullu/sports-obsreport redirects
there). (4) Contrast confirmed: the engine already pins it safely ? badminton-highlight-
indexer/pyproject.toml:68 `obsreport @ git+ssh://git@github.com/avidullu/sports-
obsreport.git@v0.1.3`. (5) No fix landed: the bare pin was introduced in commit 038424d and no
later commit touched it (git log -S on pyproject.toml); HEAD adcd206, working tree clean.
Failure scenario A (today): `pip install .[report]` in any clean env fails resolution ? the
extra is uninstallable unless the sibling checkout was pre-installed editable. Failure scenario
B (window): anyone can register `obsreport` on PyPI and publish >=0.1.3; a later `pip install
.[report]` then silently executes attacker code (sdist build hooks/import) instead of failing
loudly.
- notes: Severity calibrated down from an implied P1: the exposure is real but narrow. `report`
is an optional extra; the base install is unaffected; the collector treats obsreport as a soft
dependency that no-ops when absent (sports-data-collector/src/reporting/__init__.py:15-22); and
CI installs only `[dev]` (.github/workflows CI line 19), so no automated pipeline ever resolves
the bare name ? the confusion window only fires when a human/agent runs `pip install .[report]`
in a fresh env (in the documented sibling-editable workflow, pip sees obsreport 0.1.3 already

```

satisfied and never queries PyPI). It is still a standing security window on a generic, squatable name, plus a broken-today extra, in a repo whose sibling project already solved the identical problem. Fix (S, one line): replace pyproject.toml:39 with `obsreport @ git+ssh://git@github.com/Khelsutra/sports-obsreport.git@v0.1.3` (prefer the canonical Khelsutra org path ? the avidullu path in the engine's pin only works via GitHub's transfer redirect and is itself mildly stale; consider updating badminton-highlight-indexer/pyproject.toml:68 too). setuptools accepts PEP 508 direct references without extra config (no hatchling-style allow-direct-references needed), and since sports-data-collector is not published to PyPI, the PyPI ban on direct references does not apply. Optional hardening (owner-gated, not required for the fix): claim the `obsreport` name on PyPI with a stub to close the squatting window permanently.

R2-4 [P2/S] ? badminton-highlight-indexer

****No timeout on any serving-path ffmpeg subprocess ? one hung ffmpeg permanently wedges the single-worker job queue****

- verdict: CONFIRMED

- reasoning: Single-worker executor: backend/api/job_worker.py:47-49 (max_workers=1) with the in-code "O2 RISK" admission at :32-39 that one hung job blocks all subsequent jobs and "the executor itself cannot enforce timeouts on the Thread". Serving-path ffmpeg/ffprobe calls with NO timeout=: (compile job, POST /api/compile main.py:1594 -> _execute_compile -> compiler.compile_highlights at main.py:1686, no wrapper) backend/pipeline/stitcher/compiler.py:130 (ffprobe), :321 (per-clip libx264 encode), :494 (concat); (process job, _execute_processing main.py:1132) validators backend/pipeline/validators/format.py:43, pts_continuity.py:35, resolution_fps.py:15; backend/utls/video.py:18,:75 (chunking); backend/pipeline/segmenters/gemini.py:166; backend/pipeline/audio/hit_detector.py:53. The mitigations listed in job_worker.py:34-37 do NOT cover ffmpeg: run_bounded wraps only the Gemini generate call (backend/providers/gemini.py:233, op_timeout_sec=180); detector timeout_sec covers only GPU runner subprocesses (detectors/base.py:116, native_wasb_runner.py:134). Recovery is restart-only: fail_stale_jobs is called once at startup (main.py:2030; database.py:1122-1125) and marks ALL queued/running jobs failed. No fix landed: git head e410136; commit 24178a7 (#374) documents the O2 risk but adds no timeouts. Secondary confirmed defect: /api/source proxy warm (main.py:657 daemon thread) runs annotation_proxy.py:81 ffmpeg with no timeout; a hang leaks the _proxy_inflight dedup entry forever (main.py:607-620), permanently disabling proxy generation for that video (does not wedge the job queue).

- notes: Claim is literally true, but "permanently wedges" should be "wedges until server restart, and the restart sweep (fail_stale_jobs) then fails every queued job". Sharper failure scenario: POST /api/compile for a video whose stored local filepath sits on a network-backed mount (Google Drive streaming / SMB) ? storage.acquire_local_input is identity for local paths (main.py:1683) ? the mount stalls mid-read (or the MP4 is truncated/corrupt and ffmpeg's demuxer loops); subprocess.run at compiler.py:321 blocks forever; the compile job stays 'running'; every subsequent /api/process and /api/compile returns 202 but stays 'queued' indefinitely with zero error surfaced (drains submitted at main.py:1635 queue behind the blocked task on the max_workers=1 executor); only a full restart recovers, at the cost of failing the whole queue. Severity P2 not P0/P1: the trigger class is rare (mitigations already bound the historically hang-prone ops ? Gemini generate 180s, GPU/WSL detector timeout_sec 1800s, Cloud Run poll max_wait_sec), the risk is explicitly documented in-code, and restart recovers; but impact when triggered is a silent total outage of the job pipeline. Fix (S, mechanical): add timeout= to the ~10 serving-path ffmpeg/ffprobe subprocess.run/check_output sites (config-driven, e.g. 30-60s for ffprobe probes, stitching-level 1800s default for encode/concat/chunking) and map subprocess.TimeoutExpired to the existing clean-job-failure paths (compiler already converts CalledProcessError to a failed-job result at main.py:1687-1695; annotation_proxy already catches broadly at annotation_proxy.py:82). The broader per-job wall-clock timeout job_worker.py:38 calls a "future slice" is a separate M item and not required to close this finding. Not OWNER_GATED ? timeout defaults are an engineering choice with no spend/product implications.

R2-5 [P1/S] ? badminton-highlight-indexer

****Engine CLAUDE.md 'Current state' section is ~3 weeks false ? claims platform/owned-model work 'has not started'****

- verdict: CONFIRMED

- reasoning: Personally reproduced against the working tree and git history. (1) C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\CLAUDE.md:30-31 states "***No platform/owned-model implementation has started.**"; lines 34-37 say the "committed next PLATFORM slice = async job model" and the owned-model track "starts at a greenlit Phase-0 milestone... Do not start implementation without explicit owner approval"; lines 41-42 call `StorageBackend` + `ComputeBackend` registries "Future:". git blame shows lines 30-37 were

last written 2026-06-10 (commit 62fb2fb) and never touched since, despite CLAUDE.md itself being edited 6+ times after (e735fd4 06-13 ... af1ccbe 06-30 ? all other sections). (2) Current-tree reality contradicts every one of those claims: backend/storage/ contains a StorageBackend ABC + local/gcs/gdrive/gdrive_api/s3/capabilities implementations; backend/compute/base.py + cloud_run.py exist (ComputeBackend); backend/api/job_worker.py + tests/test_async_jobs.py exist ? the "committed next slice" async job model already shipped (8bcf6ce #339 "make /api/compile async (202 + poll)", 2026-06-22); backend/flywheel.py + backend/worker + backend/eval/served_gate_a.py exist. Owned-model track: a555887 (2026-06-20) "M2 FINALIZED ? Gen-0 weights/0.1.0-l4 pushed"; 4134721 archives DECOUPLED_COMPUTE as "DELIVERED (M0-M2 + M3.5 on GCP)"; 334ad0f (#336) M11 data-flywheel; b72e6de (#319) training worker fix. 314 commits landed since 2026-06-13, through 2026-07-01 (e410136), including a recorded launch NO-GO decision 2026-06-30 (4a7bfbe #404) ? i.e., the project reached launch-consideration, the opposite of "not started". (3) No fix landed: git log -- CLAUDE.md shows the last commit touching it (af1ccbe, 2026-06-30) added only the bit-flip-PR convention + Python 3.10 line; the Current state section is untouched since 06-10. Bonus staleness in the same file: lines 18-19 claim rally_gate.py Gate A is "not wired into production" while backend/eval/served_gate_a.py and c8a493d "served Gate-A litmus" (#401) indicate a served Gate-A path now exists.

- notes: Sharper failure scenario: CLAUDE.md is auto-loaded project memory for every fresh agent session in this heavily agent-driven repo. A fresh session reads "No platform/owned-model implementation has started" + "Do not start implementation without explicit owner approval" and will (a) tell the owner the async job model is an un-started, gated future slice and propose designing/implementing what already exists (backend/api/job_worker.py), (b) treat StorageBackend/ComputeBackend as "Future" and re-design duplicates of backend/storage/ and backend/compute/, (c) misreport project status upward (e.g., in handoffs/audits) as pre-implementation when Gen-0 weights shipped and a launch NO-GO was formally recorded 06-30. This also directly violates the workspace's own "keep it honest ? reflects real, shipped state" convention. Correctly-scoped fix (S, one doc-only PR): rewrite CLAUDE.md "Current state" (lines 29-37) to reflect reality as of 2026-07-01 ? platform slice (async jobs, storage/compute backends, capabilities/assets endpoints) SHIPPED; owned-model M0-M2+M3.5 DELIVERED on GCP, Gen-0 weights pushed; launch NO-GO 2026-06-30 with pointer to #404/docs; fix line 41-42 ("Future" ? "implemented, see backend/storage/, backend/compute/"); fix lines 18-19 Gate-A "not wired" claim against backend/eval/served_gate_a.py; re-point "committed next slice" at whatever docs/NEXT_STEPS.md currently prioritizes. Not OWNER_GATED ? this is a factual truth-sync of docs to shipped code, the same class as the repo's prior "audit truth-sync" commits (62fb2fb, 9736a3b); no spend or strategy decision required. Round-1 claim of "~3 weeks false" is accurate: section frozen 2026-06-10, contradicting implementation landed from ~2026-06-20 onward.

5. user (2026-07-02T04:46:18.664Z)

```

1      # CLAUDE.md ? project memory / agent handoff
2
3      Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
4      FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
5      repos: `sports-data-collector`, a VLC-based `rally-annotator`.
6
7      ## Read these first
8      - **`docs/README.md`** ? doc index. **`docs/CODE_MAP.md`** ? code navigation + data
9        contracts (the cold-start map; read before touching code).
10     - **Authoritative owned-model roadmap:** `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+
the
11     live `docs/NEXT_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they
12     disagree (licensing, Gemma-4 stance, base model, conversational-QA).
13     - **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`,
`docs/PMF_MARKET_RESEARCH.md`,
14     `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also
`docs/ROADMAP.md`
15     (offline-segmenter narrative) and `docs/BACKLOG.md`.
16     - **Rally-detection quality track (design merged 2026-06-13, owner-gated, implementation
in progress):**
17     `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model) ?
`docs/MULTI_SIGNAL_FUSION_PLAN.md`
18     (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally_gate.py`
19     "Gate A" **not wired into production** ? cheapest win) ? `docs/EXPERIMENT_HARNESS.md`

```

```

(A/B + shadow
20     eval so cues land flag-gated/default-OFF with zero regression) ?
`docs/ROORA_LABELING_GUIDELINES.md`
21     (v8 vendor labeling spec). **`docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.**
22     - **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
23     **checkpoints/` (latest: `2026-06-strategy-foundation/`). **Archive policy:** only
move
24     **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level ? and
**before
25     archiving, HONESTLY update the doc first** (verify claims vs code ? flip status + a
completion
26     note ? update inbound refs ? keep the lineage ? then `git mv`), per the checklist +
living
27     lifecycle register in **`docs/DOC_STATUS.md`**.
28
29     ## Current state (June 2026)
30     - **Scaffolding complete** ; strategy foundation + owned-model roadmap merged. **No
31     platform/owned-model implementation has started.** A 2026-06-10 audit-driven hardening
32     sweep landed (licensing gate, detector keying, re-ingest safety, config, eval gate,
API
33     validation, reliability, CI + test coverage) ? see git history.
34     - **Two tracks, both owner-gated:** the **committed next PLATFORM slice = async job
model**
35     (`DEFERRED_HARDENING_PLAN.md` #16, single-tenant); the **owned-model track** starts at
a
36     greenlit Phase-0 milestone (`OWNED_MODEL_IMPLEMENTATION_PLAN.md`). **Do not start
37     implementation without explicit owner approval.**
38
39     ## Architecture in one breath
40     - **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
41     / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
42     `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
43     - `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
helper).
44     - Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
45     user_version` migration ladder in `backend/infrastructure/database.py`.
46
47     ## Committed guardrails (non-negotiable)
48     - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
49     training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
50     code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
51     - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
52     is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel
53     needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).
**Exclude
54     minors** from the training pool; per-user training data needs a separate explicit opt-
in.
55     - **v1 = rally + highlight + history + correction loop** ; **gait/biometrics strictly
future**
56     (GDPR Art. 9).
57     - **Generic data schema** ; `coach ? athlete` is an **optional overlay** , not baked in.
58     - **Compute = GCP ONLY (ADR-013, 2026-06-20):** GPU training + serving run on **GCP**
(`deploy/gcp/` ;
59     Cloud Run+GPU for serving per ADR-012). **DigitalOcean is DROPPED for compute** (DO
GPU not enabled
60     on the account + not cleanly credit-funded; Paperspace is subscription-gated). Don't
re-evaluate
61     DO/Paperspace for compute without a new ADR; the `$200` DO credits go to **non-compute
only** . The

```

```

62     `deploy/digitalocean/*` scripts are kept solely as provider-agnostic Linux-GPU setup
reused by GCP.
63
64     ## Workflow conventions
65     - Branch off `master`; ONE PR per work item; NO PR stacking. Don't open a PR unless
asked.
66     - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
67     - Never push to the default branch directly; descriptive commit messages.
68     - Substantial design/project work gets a tracked project doc following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` $7,
`docs/GOLDEN_REGRESSION_FIXTURES.md`.
69     - Launch / "bit-flip" PRs (a default-flip that drives a tracked project to terminal
state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` $2 archival due-diligence on that
project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR
lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS $3), and fix any internal-link
breakage the move causes. Enforced by `.github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI
job (`python -m tools.check_md_links`).
70     - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to
the owner,
71     who has the golden corpus + labelled videos; verify pure-logic paths locally.
72

```

6. user (2026-07-02T04:46:19.403Z)

```

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
e410136 [master]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-idempotent-process e410136
[fix/340-idempotent-process] locked

```

7. user (2026-07-02T04:46:29.101Z)

```

Preparing worktree (new branch 'fix/r2-4-ffmpeg-timeouts')
branch 'fix/r2-4-ffmpeg-timeouts' set up to track 'origin/master'.
HEAD is now at e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe
details) (#413)

```

8. user (2026-07-02T04:46:34.264Z)

CODE_MAP.md

Purpose: cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.

Regenerate: rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).

? **2026-06-10 audit-hardening sweep** changed several behaviors documented below ? trust git history over stale lines until the next full regen: **add_video** is now an **UPSERT** (no FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via `segment\_watermarks`/`prune\_segments`); detector artifacts are keyed by **video_id**, not basename; **indexing.wasb** is now a real typed config (was silently dropped); **timeout_sec** defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**; **match_intervals** requires real overlap (iou>0); **run_regression** baseline lives in tracked **eval_baselines/** and **exits 3** on a missing baseline; the API validates **video_path/output_filename**. New tests: **test_api_endpoints**, **test_compiler**, **test_wasb_runner**, **test_reingest_safety**, **test_detector_keying**, **test_config_wasb**,

```
`test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`,  
`test_import_guard`, `test_cvat_robustness`).
```

****LEGEND****

- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `\$` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

Top-level tree and the ****placement rule**** for each kind of file. When unsure, prefer an existing ****registry / extension point**** (`?`) over a new top-level file.

```
| You're adding? | Put it in? | Notes |  
|---|---|---|  
| Backend / runtime code | `backend/<subsystem>/` |  
`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,  
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most  
additions are **register a class in the relevant `?` registry**, not a new module tree. |  
| **Eval LIBRARY** (importable, pure ? metrics, calibration, a scorer, a feature) |  
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.  
These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not  
scripts; do NOT move them out. |  
| **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app code) |  
**scripts/** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as  
`python scripts/<name>.py`. If it needs `load_dotenv()/sys.path` before importing backend, add  
`"scripts/<name>.py" = ["E402"]` to `ruff.toml`. |  
| **Ops / maintenance tool** (not part of the served app ? cache GC, batch admin) | `tools/` |  
Run as `python -m tools.<name>` (it's an importable package). e.g. `tools/cleanup_caches.py`.  
Keep the destructive decision path pure + unit-tested. |  
| Training code (model fitting) | `training/` | Behind the CI-enforced import wall (ADR-008) ?  
`backend.main` must not import it. Own `requirements-train.txt`. |  
| A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70% coverage  
floor**. Pure/offline + mocked (no GPU/WSL/network). |  
| A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only **terminal-  
state** (DONE/REJECTED) work moves to `docs/archives/`. |  
| A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from ruff/pytest.  
Never imported by app code. |  
| A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008: **manifest  
committed** (provenance), **weights gitignored**. |  
| Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean checkout);  
the no-regression gate reads it. |  
| **Committed regression fixture** (video-free golden) | `eval_baselines/fixtures/<slug>/` |  
Tracked, **LF-pinned** (`.gitattributes`). Synthetic Tier-0 (or owner-cleared, de-attributed  
real). Drives the `golden_fixtures` characterization gate with **no video/GPU/DB**. See  
`docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m backend.eval.golden_fixtures --freeze-  
synthetic` / `--freeze-real` (refusal-gated). |  
| Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored.** Per-  
video isolation is the tracked **PER_VIDEO_WORKSPACE.md** project (P0 helper shipped #264;  
supersedes the archived `PER_VIDEO_OUTPUT_LAYOUT`). |
```

****Canonical files that stay at the repo ROOT**** (referenced by CI / packaging / docs ? do not move):

```
`pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) · `run_all_tests.py` (CI:  
`ci.yml`) ·  
`run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) ·  
`run_eval.py`  
(grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic `setup.py` moved
```



```

to
`scripts/doctor.py` (it shadowed the build system); invoke via `python scripts/doctor.py` or
`indexer --setup`.
New standalone run-drivers go in **`scripts/`**, not the root.

```

1. PIPELINE

1A. Runtime: video file -> highlight reel

```

...
typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
                                @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
-> video_id = MD5(whole file)      @backend/utils/hashing.py::compute_video_id (single
shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint copies)
-> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
    FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity ->
SceneCut -> Blur -> Relevance    (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
    [validation FAIL -> persist status="failed", return early; report STILL emitted in
finally]
-> db.add_video(video_id, filepath, status, [r.dict()]) @backend/infrastructure/database.py
-> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
-> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
-> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
    [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without
stitching]
-> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs ? even
on validation failure / exception; NEVER raises; grafts response["reports"])
-> select segment ids -> [(start,end)] (+ stitching padding)
-> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
    per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
...
Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
...
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
-> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted via
db.add_candidate_segment]
-> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip,
prompt) ? provider_registry
-> P4 db.add_play_segment + db.link_candidate_to_segment
...
Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) ->
add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

```

1B. WASB shuttle substrate (P2 of wasb_hybrid)

9. assistant (2026-07-02T04:46:46.217Z)

Now let me read all the call sites in the worktree.

10. user (2026-07-02T04:46:48.580Z)

```

1      import logging
2      import os
3      import subprocess

```

```

4     import tempfile
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
9         ffmpeg_bin,
10        ffprobe_bin,
11    )
12
13    logger = logging.getLogger(__name__)
14
15    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
16    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
17    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
18    # backend/pipeline/stitcher/, so parents[2] == backend/.
19    _BUNDLED_FONT = (
20        Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
21    )
22
23
24    class VideoCompiler:
25        def __init__(
26            self,
27            output_dir: str,
28            end_padding: float = 1.0,
29            begin_padding: float = 0.5,
30            watermark: Optional[Any] = None,
31        ):
32            self.output_dir = output_dir
33            self.end_padding = end_padding
34            self.begin_padding = begin_padding
35            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
36            # normalized to a dict (or None) so this module stays config-package-agnostic.
37            self.watermark = self._normalize_watermark(watermark)
38            os.makedirs(output_dir, exist_ok=True)
39
40        @staticmethod
41        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
42            if watermark is None:
43                return None
44            if hasattr(watermark, "model_dump"):
45                return watermark.model_dump()
46            if isinstance(watermark, dict):
47                return dict(watermark)
48            return None
49
50        @staticmethod
51        def _ff_quote(path: str) -> str:
52            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
53            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
54            slashes (Windows-friendly), escaped single quotes, AND escaped colons.
55
56            The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
57            option parser treats ':' as the option separator, so a Windows drive-letter
58            path like `C:/dir/font.ttf` otherwise truncates the operand at `C` and
59            the encode fails with "Invalid argument". This silently disabled the
60            on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
61            return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
62
63        @staticmethod
64        def _parse_time(val: Union[int, float, str]) -> float:
65            """Normalizes a timestamp value to float seconds.

```

```

66         Handles: float/int (pass-through), plain numeric strings ("12.4"),
67         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
68         This mirrors the parse_time logic used in gemini.py to ensure
69         the stitcher can always consume whatever format the indexer produced.""
70         if isinstance(val, (int, float)):
71             return float(val)
72         if isinstance(val, str):
73             val = val.strip()
74             if ":" in val:
75                 # Handle MM:SS, HH:MM:SS, etc.
76                 parts = val.split(":")
77                 parts.reverse()
78                 total = 0.0
79                 for i, p in enumerate(parts):
80                     try:
81                         total += float(p) * (60**i)
82                     except ValueError:
83                         pass
84                 return total
85             try:
86                 return float(val)
87             except ValueError:
88                 logger.warning(
89                     f"Could not parse timestamp value '{val}' as a number. Defaulting to
0.0."
90                 )
91                 return 0.0
92         logger.warning(
93             f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0."
94         )
95         return 0.0
96
97     @staticmethod
98     def _merge_overlapping(
99         segments: List[Tuple[float, float]],
100     ) -> List[Tuple[float, float]]:
101         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
102
103         A highlight reel must never replay the same footage. The same rally can land in
104         the segment list more than once ? e.g. two detector runs accumulated for one
105         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
106         or
107         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
108         rally
109         twice. Merging any range that overlaps or abuts the previous one collapses true
110         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
111         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
112         """
113         parsed = []
114         for raw_s, raw_e in segments:
115             s = VideoCompiler._parse_time(raw_s)
116             e = VideoCompiler._parse_time(raw_e)
117             if e > s:
118                 parsed.append((s, e))
119         parsed.sort()
120         merged: List[Tuple[float, float]] = []
121         for s, e in parsed:
122             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
123                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
124             else:
125                 merged.append((s, e))
126         return merged

```

```

127     """Probes the video file to get its total duration in seconds.
128     Returns 0.0 if it cannot be determined."""
129     try:
130         result = subprocess.run(
131             [
132                 ffprobe_bin(),
133                 "-v",
134                 "error",
135                 "-show_entries",
136                 "format=duration",
137                 "-of",
138                 "default=noprint_wrappers=1:nokey=1",
139                 video_path,
140             ],
141             capture_output=True,
142             text=True,
143             check=True,
144         )
145         return float(result.stdout.strip())
146     except Exception as e:
147         logger.warning(f"Could not determine video duration via ffprobe: {e}")
148         return 0.0
149
150     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
151         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
152         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
153         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
154         because the mark is already baked into every clip identically."""
155         wm = self.watermark
156         if not wm or not wm.get("enabled"):
157             return None
158
159         text = (wm.get("text") or "").strip()
160         logo_path = (wm.get("logo_path") or "").strip()
161         if logo_path and not os.path.exists(logo_path):
162             logger.warning(
163                 f"[watermark] logo not found: {logo_path} ? skipping logo overlay."
164             )
165             logo_path = ""
166         if not text and not logo_path:
167             return None
168
169         margin = int(wm.get("margin", 24))
170         opacity = float(wm.get("opacity", 0.85))
171         position = str(wm.get("position", "bottom-right"))
172         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
173         dt_xy = {
174             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
175             "bottom-left": f"x={margin}:y=h-th-{margin}",
176             "top-right": f"x=w-tw-{margin}:y={margin}",
177             "top-left": f"x={margin}:y={margin}",
178         }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
179         ov_xy = {
180             "bottom-right": f"W-w-{margin}:H-h-{margin}",
181             "bottom-left": f"{margin}:H-h-{margin}",
182             "top-right": f"W-w-{margin}:{margin}",
183             "top-left": f"{margin}:{margin}",
184         }.get(position, f"W-w-{margin}:H-h-{margin}")
185
186         drawtext = ""
187         if text:
188             ratio = float(wm.get("font_size_ratio") or 0.035)
189             divisor = max(1, round(1.0 / ratio))
190             # textfile= sidesteps the notorious drawtext text= escaping for arbitrary

```

```

strings.
191         tf_path = os.path.join(tmp_dir, "watermark.txt")
192         with open(tf_path, "w", encoding="utf-8") as fh:
193             fh.write(text)
194         # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
195         # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
196         font_path = (wm.get("font_path") or "").strip()
197         if not font_path and _BUNDLED_FONT.exists():
198             font_path = str(_BUNDLED_FONT)
199         if font_path:
200             font_opt = f"fontfile='{self._ff_quote(font_path)}'"
201         else:
202             font_opt = f"font='{wm.get('font', 'Sans')}'"
203         box = ""
204         if wm.get("box", True):
205             box = f"box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
206         drawtext = (
207             f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
208             f"fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
209         )
210
211         if logo_path and drawtext:
212             return
213         f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
214         if logo_path:
215             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
216         return drawtext
217
218     def _trim_one_segment(
219         self,
220         idx: int,
221         raw_start: Union[int, float, str],
222         raw_end: Union[int, float, str],
223         segments: List[Tuple[float, float]],
224         video_duration: float,
225         tmp_dir: str,
226         raw_video_path: str,
227         watermark_filter: Optional[str],
228     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
229         """Trim a single segment into its own clip file (extracted verbatim from the
230         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
231         success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
232         None and the (idx, start, end, reason) record to record in skipped_segments."""
233         # Normalize timestamps to float seconds ? handles float, int,
234         # plain numeric strings, and colon-separated formats like "MM:SS"
235         start = self._parse_time(raw_start)
236         end = self._parse_time(raw_end)
237         segment_label = (
238             f"Segment #{idx + 1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
239         )
240
241         # Validate the segment times
242         if start < 0:
243             logger.warning(
244                 f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to
0."
245             )
246             start = 0.0
247
248         if start >= end:
249             logger.error(
250                 f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
Skipping invalid segment."
251             )

```

```

251         return None, (idx, start, end, "start >= end")
252
253     # Check if segment extends beyond video duration
254     if video_duration > 0:
255         if start >= video_duration:
256             logger.error(
257                 f"{segment_label}: start_time ({start:.2f}s) is beyond the video
duration ({video_duration:.2f}s). "
258                 f"This segment cannot be extracted ? the video ran out. Skipping."
259             )
260             return None, (idx, start, end, "start beyond video end")
261
262         if end > video_duration:
263             original_end = end
264             end = video_duration
265             logger.warning(
266                 f"{segment_label}: end_time ({original_end:.2f}s) exceeds video
duration ({video_duration:.2f}s). "
267                 f"Clamping end_time to {video_duration:.2f}s. The segment may be
truncated."
268             )
269
270         clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
271
272         # Precise sub-second trim using fast re-encoding to preserve stream headers
273         padded_start = max(0.0, start - self.begin_padding)
274         padded_end = end + self.end_padding
275
276         # Clamp padded_end to video duration if known
277         if video_duration > 0 and padded_end > video_duration:
278             padded_end = video_duration
279             logger.info(
280                 f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds
video duration. "
281                 f"Clamping to {video_duration:.2f}s (end padding partially applied)."
282             )
283
284         trim_duration = padded_end - padded_start
285         logger.info(
286             f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
287         )
288
289         base_cmd = [
290             ffmpeg_bin(),
291             "-y",
292             "-ss",
293             str(round(padded_start, 3)),
294             "-to",
295             str(round(padded_end, 3)),
296             "-i",
297             raw_video_path,
298             "-c:v",
299             "libx264",
300             "-preset",
301             "superfast",
302             "-crf",
303             "22",
304             "-c:a",
305             "aac",
306             "-b:a",
307             "128k",
308         ]
309         vf_args = ["-vf", watermark_filter] if watermark_filter else []
310

```

```

311     # Try with the watermark first; if that encode fails (e.g. a bad font/logo
312     # path), retry once WITHOUT it so a branding misconfig can never nuke the
313     # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
314     attempts = [vf_args, []] if vf_args else [[]]
315     produced = False
316     last_exc: Optional[subprocess.CallProcessError] = None
317     cmd = base_cmd + [clip_path]
318     for attempt_vf in attempts:
319         cmd = base_cmd + attempt_vf + [clip_path]
320         try:
321             subprocess.run(cmd, capture_output=True, check=True)
322         except subprocess.CallProcessError as e:
323             last_exc = e
324             if attempt_vf and vf_args:
325                 _err = (
326                     e.stderr.decode(errors="replace").strip()[-300:]
327                     if getattr(e, "stderr", None)
328                     else ""
329                 )
330                 logger.warning(
331                     f"{segment_label}: watermark encode failed; retrying without it.
332 "
333                     f"ffmpeg: {_err or '(no stderr captured)'}"
334                 )
335                 continue
336             break
337         if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
338             produced = True
339             if not attempt_vf and vf_args:
340                 logger.warning(
341                     f"{segment_label}: encoded WITHOUT the watermark (the watermark
342 filter failed)."
343                 )
344             break
345         last_exc = None # ffmpeg succeeded but produced nothing
346         break
347     if produced:
348         return clip_path, None
349     elif last_exc is not None:
350         stderr_msg = last_exc.stderr.decode(errors="replace").strip()
351         logger.error(f"{segment_label}: FFmpeg trim failed.")
352         print(f"  Command: {' '.join(cmd)}")
353         print(
354             f"  FFmpeg stderr: {stderr_msg[-500:]}")
355         ) # Last 500 chars to avoid flooding
356         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
357     else:
358         logger.error(
359             f"{segment_label}: FFmpeg exited successfully but output clip is missing
360 or empty. Skipping."
361         )
362         return None, (idx, start, end, "empty output clip")
363
364 def compile_highlights(
365     self,
366     raw_video_path: str,
367     segments: List[Tuple[float, float]],
368     output_filename: str,
369 ) -> str:
370     """
371     Extracts selected segments from a raw video and stitches them together
372     into a seamless highlights file, applying end_padding to prevent abrupt endings.
373
374     .. warning::

```

```

373         This method uses a two-stage codec contract: per-clip re-encodes must use
374         IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
375         concat can stream-copy (`-c copy`) without re-encoding. Changing the
376         clip encode settings without updating the concat assumptions will produce
377         broken output.
378         """
379         if not segments:
380             raise ValueError("No highlight segments provided to compile.")
381
382         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
383         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
384         original_count = len(segments)
385         segments = self._merge_overlapping(segments)
386         if len(segments) < original_count:
387             logger.info(
388                 "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
389                 %d.",
390                 original_count - len(segments),
391                 original_count,
392                 len(segments),
393             )
394         if not segments:
395             raise ValueError("No valid (start < end) highlight segments after merge.")
396
397         if not os.path.exists(raw_video_path):
398             raise FileNotFoundError(
399                 f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
400             )
401
402         # Defense-in-depth: never let the output name escape output_dir (the API also
403         # validates this at the request boundary).
404         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
405
406         # Probe video duration so we can detect out-of-range segments
407         video_duration = self._get_video_duration(raw_video_path)
408         if video_duration > 0:
409             logger.info(f"Source video duration: {video_duration:.2f}s")
410         else:
411             logger.warning(
412                 "Video duration unknown. Cannot validate segment boundaries against
413                 video length."
414             )
415
416         # We will create temporary clips and stitch them using FFmpeg concat demuxer
417         temp_clips = []
418         skipped_segments = []
419
420         # Create a temp directory within output_dir
421         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
422             logger.info(
423                 f"Trimming {len(segments)} segments
424                 (begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
425             )
426             # Build the (optional) watermark filtergraph once; it is applied during
427             every
428             # per-clip re-encode so the mark is baked into each clip identically.
429             watermark_filter = self._watermark_filter(tmp_dir)
430             if watermark_filter:
431                 logger.info("[watermark] applying branding overlay to each clip.")
432             for idx, (raw_start, raw_end) in enumerate(segments):
433                 clip_path, skip_record = self._trim_one_segment(
434                     idx,
435                     raw_start,
436                     raw_end,
437                     segments,

```



```

434         video_duration,
435         tmp_dir,
436         raw_video_path,
437         watermark_filter,
438     )
439     if clip_path is not None:
440         temp_clips.append(clip_path)
441     if skip_record is not None:
442         skipped_segments.append(skip_record)
443
444     # Report summary of skipped segments
445     if skipped_segments:
446         logger.info(
447             f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
448         )
449         for seg_idx, seg_start, seg_end, reason in skipped_segments:
450             logger.info(
451                 f"    - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
452             )
453         logger.info("")
454
455     if not temp_clips:
456         raise RuntimeError(
457             f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
458             f"All {len(segments)} segments were skipped. "
459             f"Check the errors above for details. "
460             f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
461         )
462
463     logger.info(
464         f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
465     )
466
467     # Create a list file for FFmpeg concat demuxer
468     list_file_path = os.path.join(tmp_dir, "clips.txt")
469     # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
470
471     # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
472     with open(list_file_path, "w", encoding="utf-8") as f:
473         for clip in temp_clips:
474             # Escape single quotes in path
475             safe_path = clip.replace("'", "'\\'")
476             f.write(f"file '{safe_path}'\n")
477
478     logger.info("Stitching clips together...")
479     # Stitch them together without re-encoding (since they were encoded with
identical settings)
480     concat_cmd = [
481         ffmpeg_bin(),
482         "-y",
483         "-f",
484         "concat",
485         "-safe",
486         "0",
487         "-i",
488         list_file_path,
489         "-c",
490         "copy",
491         output_path,
492     ]

```

```

493         try:
494             subprocess.run(concat_cmd, capture_output=True, check=True)
495
496         # Final verification
497         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
498             raise RuntimeError(
499                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
500                 f"but output file is missing or empty: {output_path}"
501             )
502
503         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
504         logger.info(
505             f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
506 MB)"
507         )
508         return output_path
509     except subprocess.CalledProcessError as e:
510         stderr_msg = e.stderr.decode(errors="replace").strip()
511         raise RuntimeError(
512             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
513             f"Command: {' '.join(concat_cmd)}\n"
514             f"FFmpeg stderr: {stderr_msg[-500:]}"
515         ) from e

```

11. user (2026-07-02T04:46:49.665Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
2     P3).
3
4     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
5     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
6     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
7     re-claimed when due ? with no central scheduler (the DB is the clock).
8
9     The monkeypatch contract these functions rely on (and that tests depend on):
10    every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
11    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
12    `backend.main` module object at call-time (`import backend.main as _main`), never as a
13    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
14    ...) still intercepts the inter-function calls (e.g. a parked job arms the *patched*
15    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
16    lazily
17    inside the functions also avoids the circular import at module load (main.py imports
18    this
19    module; this module must not import main at import time).
20    """
21
22    import logging
23    import threading
24    import traceback
25    from concurrent.futures import ThreadPoolExecutor
26    from typing import Any, Dict, Optional
27
28    logger = logging.getLogger(__name__)
29
30    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
31    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
32    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
33    # already parallelize their AI handoff internally via their own pools.
34    #
35    # ? O2 RISK (CODE_CLEANUP_AUDIT $3c): a single worker means ONE stuck or hung job
36    # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
37    # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
38    # - ExecutionPolicy op_timeout_sec kills hung generate/processing calls

```

```

36 # - detector timeout_sec kills hung WSL/GPU calls
37 # - Cloud Run poll has a max_wait_sec bound
38 # A future slice should add a per-job wall-clock timeout (the executor itself
39 # cannot enforce timeouts on the Thread).
40 _job_executor: Optional[ThreadPoolExecutor] = None
41
42
43 def _get_executor() -> ThreadPoolExecutor:
44     """Lazy module-global executor; tests monkeypatch this for inline execution."""
45     global _job_executor
46     if _job_executor is None:
47         _job_executor = ThreadPoolExecutor(
48             max_workers=1, thread_name_prefix="jobworker"
49         )
50     return _job_executor
51
52
53 class JobWaiting(Exception):
54     """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
55     WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
56     NOT a failure. The worker persists `next_poll_at` + `progress` (via
57     `set_job_waiting`),
58     releases the thread, and re-claims the job when due (`claim_due_job`). No work to
59     date
60     is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
61     contract.
62
63     The wiring is additive: the production segmenter path never raises this today (zero
64     behaviour change), so a job runs exactly as before. The hook is what a later slice
65     (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
66     """
67
68     def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
69         super().__init__(f"job parked for {delay_sec:.0f}s")
70         self.delay_sec = max(0.0, float(delay_sec))
71         self.progress = progress
72
73
74 def _job_to_request(job: Dict[str, Any]):
75     """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
76     re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
77     the original submit. The row stores exactly the ProcessRequest fields."""
78     import backend.main as _main
79
80     return _main.ProcessRequest(
81         video_path=job["video_path"],
82         player_pool=job.get("player_pool"),
83         max_frames=job.get("max_frames"),
84         segmenter=job.get("segmenter"),
85         # v8 compute-picker: the worker runs _execute_processing ?
86         _maybe_dispatch_remote, which
87         # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
88         # only read off
89         # the job row for analytics). NULL ? server default = pre-picker behaviour.
90         compute=job.get("compute"),
91     )
92
93
94 def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
95     """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
96     its
97     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
98
99     Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
100     means

```

```

94     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
95     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
96     job self-scheduled and will be re-claimed when `next_poll_at` is due.
97     """
98     import backend.main as _main
99
100     job_id = job["id"]
101     # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
102     # original /api/process pipeline). Both record their terminal state the same way;
only process
103     # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
104     is_compile = job.get("kind") == "compile"
105     try:
106         if is_compile:
107             result = _main._execute_compile(
108                 config,
109                 db,
110                 job,
111                 progress=lambda stage: db.set_job_progress(job_id, stage),
112             )
113         else:
114             req = _main._job_to_request(job)
115             result = _main._execute_processing(
116                 config,
117                 db,
118                 req,
119                 progress=lambda stage: db.set_job_progress(job_id, stage),
120             )
121             if result.get("video_id"):
122                 db.set_job_video_id(job_id, result["video_id"])
123             db.mark_job_done(job_id, result)
124             if not is_compile:
125                 _maybe_record_job_cost(
126                     job, result, config, db
127                 ) # M8 cost ledger (default-OFF; process jobs only)
128                 _maybe_run_flywheel(
129                     job, result, config
130                 ) # M11 data-flywheel (process jobs only)
131     except _main.JobWaiting as w:
132         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
133         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
134         db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
135     except Exception as e:
136         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
137         db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
138
139
140     def _maybe_record_job_cost(
141         job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
142     ) -> None:
143         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
144         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
145         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
146         persist it.
147
148         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
149         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
150         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook

```

```

151         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
152         With the placeholder pricebook the cost is 0 regardless.
153
154         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
155         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
156
157         billing_cfg = getattr(config, "billing", None)
158         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
159             return
160         try:
161             usage = (result or {}).get("usage") or {}
162             db.record_job_cost(
163                 job["id"],
164                 usage=usage,
165                 pricebook_version=billing_cfg.pricebook_version,
166                 gpu_type=usage.get("gpu_type"),
167                 compute_backend=(result or {}).get("compute_backend"),
168                 owner_id=job.get("owner_id"),
169                 margin=billing_cfg.margin,
170             )
171             logger.info(
172                 "Recorded cost for job %s (pricebook=%s).",
173                 job["id"],
174                 billing_cfg.pricebook_version,
175             )
176         except Exception as e: # never wedge a completed job on bookkeeping
177             logger.warning(
178                 "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
179             )
180
181
182     def _maybe_run_flywheel(
183         job: Dict[str, Any], result: Dict[str, Any], config: Any
184     ) -> None:
185         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
186         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
187         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
188         land), so this captures **nothing** in production. When activated it captures the
consented job's
189         artifacts into the per-video workspace (? later labeling ? the golden training set).
190
191         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
192         row (carries ``owner_id``); ``result`` is the pipeline output."""
193
194         flywheel_cfg = getattr(config, "flywheel", None)
195         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
196             return
197         try:
198             from backend import flywheel
199
200             flywheel.run_for_job(job, result, config)
201         except Exception as e: # never wedge a completed job on the flywheel
202             logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
203
204
205     def _drain_due_jobs(config: Any, db: Any) -> int:
206         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
207         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

```

```

208
209     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
210     up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
211     a single-worker box keeps making progress with no central timer (the DB is the
clock).
212     Returns the number of jobs that reached a terminal/parked state this tick.
213     """
214     import backend.main as _main
215
216     ran = 0
217     while True:
218         job = db.claim_due_job()
219         if job is None:
220             break
221         ran += 1
222         _main._run_claimed_job(job, config, db)
223     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
224     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
225     # on the next submit/drain. Best-effort ? never raises into the worker.
226     _main._schedule_next_drain_if_waiting(config, db)
227     return ran
228
229
230 def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
231     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
232     self-scheduled job resumes with no further client submit. The drain itself re-checks
233     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
234     import backend.main as _main
235
236     try:
237         delay = db.seconds_until_next_due()
238     except Exception as e: # pragma: no cover - defensive; never wedge the worker
239         logger.error(f"Could not compute next-due delay: {e}")
240         return
241     if delay is None:
242         return # nothing waiting
243     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
244     _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
245
246
247 def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
248     """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
249     scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
250     monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
251     import backend.main as _main
252
253     timer = threading.Timer(
254         delay_sec,
255         lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
256     )
257     timer.daemon = True
258     timer.start()
259
260
261 #: A parked job further out than this is re-checked by a capped wake rather than one
long

```

```
262     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
263     _MAX_DRAIN_WAKE_SEC = 60.0
264
```

12. user (2026-07-02T04:46:58.734Z)

```
1     import json
2     import os
3     import subprocess
4     from typing import Any
5
6     from backend.pipeline.validators.base import ValidationResult, VideoValidator
7     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
8
9
10    class FormatValidator(VideoValidator):
11        @property
12        def name(self) -> str:
13            return "format_check"
14
15        @property
16        def description(self) -> str:
17            return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
18
19        def validate(
20            self, video_path: str, config: Any, context: dict[str, Any]
21        ) -> ValidationResult:
22            if not os.path.exists(video_path):
23                return ValidationResult(
24                    validator_name=self.name,
25                    passed=False,
26                    message=f"File not found: {video_path}",
27                )
28
29            cmd = [
30                ffprobe_bin(),
31                "-v",
32                "error",
33                "-select_streams",
34                "v:0",
35                "-show_format",
36                "-show_streams",
37                "-of",
38                "json",
39                video_path,
40            ]
41
42            try:
43                result = subprocess.run(cmd, capture_output=True, text=True, check=True)
44                info = json.loads(result.stdout)
45
46                if not info.get("streams") or not info.get("format"):
47                    return ValidationResult(
48                        validator_name=self.name,
49                        passed=False,
50                        message="Failed to retrieve valid video streams from the file.",
51                    )
52
53                # Store the metadata in context for subsequent validators to utilize
54                context["ffprobe_meta"] = info
55
56                stream = info["streams"][0]
57                fmt = info["format"]
58
```

```

59         details = {
60             "format_name": fmt.get("format_name"),
61             "duration": float(fmt.get("duration", 0)),
62             "size_bytes": int(fmt.get("size", 0)),
63             "codec_name": stream.get("codec_name"),
64         }
65
66         # Check if container name includes mp4
67         if "mp4" not in fmt.get("format_name", "").lower():
68             return ValidationResult(
69                 validator_name=self.name,
70                 passed=False,
71                 message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
72                 details=details,
73             )
74
75         return ValidationResult(
76             validator_name=self.name,
77             passed=True,
78             message="Video format is valid MP4 container.",
79             details=details,
80         )
81
82     except subprocess.CalledProcessError as e:
83         return ValidationResult(
84             validator_name=self.name,
85             passed=False,
86             message="Video stream is corrupted or unreadable by ffprobe.",
87             details={"error": e.stderr},
88         )
89     except Exception as e:
90         return ValidationResult(
91             validator_name=self.name,
92             passed=False,
93             message=f"Unexpected error running ffprobe: {str(e)}",
94         )
95

```

13. user (2026-07-02T04:46:59.638Z)

```

1     import subprocess
2     from typing import Any
3
4     from backend.pipeline.validators.base import ValidationResult, VideoValidator
5     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
6
7
8     class PTSContinuityValidator(VideoValidator):
9         @property
10         def name(self) -> str:
11             return "pts_continuity_check"
12
13         @property
14         def description(self) -> str:
15             return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
timeline gaps, jumps, or editing splices."
16
17         def validate(
18             self, video_path: str, config: Any, context: dict[str, Any]
19         ) -> ValidationResult:
20             # Command to extract packet PTS and flags (extremely fast, works on H.264 &
H.265)
21             cmd = [
22                 ffprobe_bin(),

```



```

23         "-v",
24         "error",
25         "-select_streams",
26         "v:0",
27         "-show_entries",
28         "packet=pts_time,flags",
29         "-of",
30         "csv=p=0",
31         video_path,
32     ]
33
34     try:
35         result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36         output = result.stdout.strip()
37
38         if not output:
39             return ValidationResult(
40                 validator_name=self.name,
41                 passed=False,
42                 message="No packet timestamps could be extracted from the video
stream.",
43             )
44
45         timestamps = []
46         for line in output.split("\n"):
47             line = line.strip()
48             if line:
49                 parts = line.split(",")
50                 if len(parts) == 2:
51                     pts_str, flags = parts
52                     if "K" in flags: # Check if keyframe
53                         try:
54                             timestamps.append(float(pts_str))
55                         except ValueError:
56                             continue
57
58         if len(timestamps) < 2:
59             return ValidationResult(
60                 validator_name=self.name,
61                 passed=True,
62                 message="Video is too short to perform a PTS continuity check.",
63                 details={"keyframes_count": len(timestamps)},
64             )
65
66         # Analyze gaps
67         max_gap = 0.0
68         backward_jumps = 0
69         gaps = []
70
71         for i in range(1, len(timestamps)):
72             gap = timestamps[i] - timestamps[i - 1]
73             gaps.append(gap)
74             if gap < 0:
75                 backward_jumps += 1
76             if gap > max_gap:
77                 max_gap = gap
78
79         # A continuous GoPro recording usually has keyframe intervals (GOP size)
80         # of 1.0 to 4.0 seconds. A gap larger than the configured threshold
81         # (or a backward jump) indicates edited/spliced content.
82         details = {
83             "keyframes_count": len(timestamps),
84             "max_keyframe_gap_seconds": round(max_gap, 2),
85             "backward_jumps_detected": backward_jumps,
86             "first_keyframe": timestamps[0],

```

```

87         "last_keyframe": timestamps[-1],
88     }
89
90     if backward_jumps > 0:
91         return ValidationResult(
92             validator_name=self.name,
93             passed=False,
94             message="Video integrity compromise: Backward timeline jumps
(splicing) detected.",
95             details=details,
96         )
97
98     max_gap_threshold = config.validation.pts_max_gap_seconds
99     if max_gap > max_gap_threshold:
100         return ValidationResult(
101             validator_name=self.name,
102             passed=False,
103             message=f"Video integrity compromise: Detected a timestamp gap of
{round(max_gap, 2)}s, indicating edited or missing content.",
104             details=details,
105         )
106
107     return ValidationResult(
108         validator_name=self.name,
109         passed=True,
110         message="PTS timeline is continuous with no abnormal gaps or backward
jumps.",
111         details=details,
112     )
113
114     except subprocess.CalledProcessError as e:
115         return ValidationResult(
116             validator_name=self.name,
117             passed=False,
118             message="Failed to execute ffprobe PTS analysis.",
119             details={"error": e.stderr},
120         )
121     except Exception as e:
122         return ValidationResult(
123             validator_name=self.name,
124             passed=False,
125             message=f"Unexpected error in PTS validation: {str(e)}",
126         )
127

```

14. user (2026-07-02T04:47:01.088Z)

```

1     import json
2     import logging
3     import subprocess
4     from typing import Any
5
6     from backend.pipeline.validators.base import ValidationResult, VideoValidator
7     from backend.utils.ffmpeg import ffprobe_bin
8
9     logger = logging.getLogger(__name__)
10
11
12     def _probe_meta(video_path: str) -> dict | None:
13         """Re-probe ffprobe metadata as a fallback when format_check hasn't run first."""
14         try:
15             result = subprocess.run(
16                 [
17                     ffprobe_bin(),
18                     "-v",

```

```

19         "error",
20         "-select_streams",
21         "v:0",
22         "-show_format",
23         "-show_streams",
24         "-of",
25         "json",
26         video_path,
27     ],
28     capture_output=True,
29     text=True,
30     check=True,
31 )
32     return json.loads(result.stdout)
33 except (subprocess.CalledProcessError, json.JSONDecodeError, OSError):
34     return None
35
36
37 class ResolutionFPSValidator(VideoValidator):
38     @property
39     def name(self) -> str:
40         return "resolution_fps_check"
41
42     @property
43     def description(self) -> str:
44         return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
45
46     def validate(
47         self, video_path: str, config: Any, context: dict[str, Any]
48     ) -> ValidationResult:
49         meta = context.get("ffprobe_meta")
50         if not meta or not meta.get("streams"):
51             # Fallback: re-probe instead of hard-failing (format_check may not
52             # have run, e.g. if validator ordering changed).
53             logger.warning(
54                 "resolution_fps_check: ffprobe_meta not in shared context ? "
55                 "re-probing. Ensure format_check runs before resolution_fps_check "
56                 "in the validator registry."
57             )
58             meta = _probe_meta(video_path)
59         if not meta or not meta.get("streams"):
60             return ValidationResult(
61                 validator_name=self.name,
62                 passed=False,
63                 message="Video metadata is missing. Run format_check validator first.",
64             )
65
66         stream = meta["streams"][0]
67         width = int(stream.get("width", 0))
68         height = int(stream.get("height", 0))
69
70         # Parse frame rate (e.g. "30/1" or "30000/1001")
71         fps_str = stream.get("r_frame_rate", "0/1")
72         try:
73             num, den = map(int, fps_str.split("/"))
74             fps = num / den if den != 0 else 0
75         except ValueError:
76             fps = 0.0
77
78         details = {"width": width, "height": height, "fps": round(fps, 2)}
79
80         # Check bounds from typed config (AppConfig.validation)
81         val_cfg = config.validation
82         min_w = val_cfg.min_resolution_width

```

```

83         min_h = val_cfg.min_resolution_height
84         min_fps = val_cfg.min_fps
85
86         if width < min_w or height < min_h:
87             return ValidationResult(
88                 validator_name=self.name,
89                 passed=False,
90                 message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
91                 details=details,
92             )
93
94         if fps < min_fps:
95             return ValidationResult(
96                 validator_name=self.name,
97                 passed=False,
98                 message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
99                 details=details,
100             )
101
102         return ValidationResult(
103             validator_name=self.name,
104             passed=True,
105             message="Video resolution and frame rate meet minimum standards.",
106             details=details,
107         )
108

```

15. user (2026-07-02T04:47:02.084Z)

```

1     import logging
2     import subprocess
3     from typing import Optional
4
5     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
6         ffmpeg_bin,
7         ffprobe_bin,
8     )
9
10    logger = logging.getLogger(__name__)
11
12
13    def get_video_duration(video_path: str) -> float:
14        """Returns the exact duration of a video file in seconds using ffprobe.
15        Returns 0.0 on failure."""
16        try:
17            duration_output = (
18                subprocess.check_output(
19                    [
20                        ffprobe_bin(),
21                        "-v",
22                        "error",
23                        "-show_entries",
24                        "format=duration",
25                        "-of",
26                        "default=noprint_wrappers=1:nokey=1",
27                        video_path,
28                    ]
29                )
30                .decode()
31                .strip()
32            )
33            return float(duration_output)
34        except Exception as e:

```

```

35         logger.error(f"Could not detect video duration for {video_path}: {e}")
36         return 0.0
37
38
39     def extract_video_chunk(
40         video_path: str,
41         start_time: float,
42         duration: float,
43         output_path: str,
44         reencode: bool = False,
45         scale_width: Optional[int] = None,
46     ) -> bool:
47         """Extracts a chunk from a video file.
48
49         Args:
50             reencode: If True, re-encodes with libx264 (slower but frame-accurate).
51                       If False, uses stream-copy (fast but cuts at nearest keyframe).
52                       Use True for short clips where keyframe drift matters.
53             scale_width: If set (requires reencode), downscale to this width (height auto,
54                          even). Useful for high-res sources headed to an API that analyses at
55                          low res anyway ? shrinks uploads with no analysis loss.
56
57         """
58         cmd = [
59             ffmpeg_bin(),
60             "-y",
61             "-ss",
62             str(start_time),
63             "-t",
64             str(duration),
65             "-i",
66             video_path,
67         ]
68         if reencode:
69             if scale_width:
70                 cmd += ["-vf", f"scale={int(scale_width)}:-2"]
71             cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
72         else:
73             cmd += ["-c", "copy"]
74         cmd.append(output_path)
75         try:
76             subprocess.run(
77                 cmd, check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
78             )
79             return True
80         except Exception as e:
81             logger.error(
82                 f"Failed to extract chunk {start_time}s - {start_time + duration}s: {e}"
83             )
84             return False

```

16. user (2026-07-02T04:47:09.368Z)

```

120         return Failure(str(e))
121
122         return sport_profile, provider
123
124     def _maybe_trim_video(
125         self, video_id: str, video_path: str
126     ) -> Tuple[str, Callable[[], None]]:
127         """Optionally trim the video to indexing.debug_duration seconds (debug feature).
128
129         Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes
130         any
131         temp trimmed file created here.

```

```

131         """
132         # A: Check for debug_duration to trim the video first
133         debug_duration = self.config.indexing.debug_duration
134         video_to_upload = video_path
135         is_trimmed = False
136         temp_trimmed_path = None
137
138         if debug_duration:
139             logger.info(
140                 f"DEBUG FEATURE: Trimming video to first {debug_duration} seconds for
fast processing..."
141             )
142             base = output_base(self.config)
143             temp_trimmed_path = os.path.join(base, f"temp_trimmed_{video_id}.mp4")
144             os.makedirs(base, exist_ok=True)
145             if os.path.exists(temp_trimmed_path):
146                 try:
147                     os.remove(temp_trimmed_path)
148                 except Exception:
149                     pass
150
151             cmd = [
152                 ffmpeg_bin(),
153                 "-y",
154                 "-ss",
155                 "0",
156                 "-t",
157                 str(debug_duration),
158                 "-i",
159                 video_path,
160                 "-c",
161                 "copy",
162                 temp_trimmed_path,
163             ]
164             logger.debug(f"Running command: {' '.join(cmd)}")
165             try:
166                 subprocess.run(
167                     cmd,
168                     check=True,
169                     stdout=subprocess.DEVNULL,
170                     stderr=subprocess.DEVNULL,
171                 )
172                 logger.info(
173                     f"Trimmed video successfully created at: {temp_trimmed_path}"
174                 )
175                 video_to_upload = temp_trimmed_path
176                 is_trimmed = True
177             except Exception as e:
178                 logger.warning(
179                     f"Failed to trim video using ffmpeg: {e}. Falling back to full
video."
180                 )
181
182         # Helper to clean up local temp files
183         def cleanup_temp_files():
184             if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
185                 try:
186                     os.remove(temp_trimmed_path)
187                 except Exception:
188                     pass
189
190         return video_to_upload, cleanup_temp_files
191
192     def _process_chunked(
193         self,

```

```

194         video_id: str,
195         video_to_upload: str,
196         sport_profile,
197         provider,
198         video_duration: float,
199         chunk_duration: float,
200         chunk_overlap: float,
201         ai_scale_width: int,
202         chunks_dir: str,
203         cleanup_temp_files: Callable[[], None],
204     ) -> List[Dict[str, Any]]:
205         """--- CHUNKED PROCESSING --- (long videos): split into overlapping chunks,
206         analyze them in parallel, offset timestamps back to full-video coordinates."""
207         ai_segments: List[Dict[str, Any]] = []
208         step = chunk_duration - chunk_overlap
209         chunk_starts = []

```

17. user (2026-07-02T04:47:10.883Z)

```

1     """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).
2
3     Stages 0?1 of docs/AUDIO_RALLY_DETECTION_PLAN.md: extract the audio track, build a
4     spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient
5     a racket?shuttle contact makes), threshold it adaptively, and return (timestamp,
6     strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
7     only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion
8     (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.
9     """
10
11     from __future__ import annotations
12
13     import os
14     import subprocess
15     import tempfile
16     import wave
17     from dataclasses import dataclass
18     from typing import List, Optional, Tuple
19
20     import numpy as np
21
22     from backend.utils.ffmpeg import ffmpeg_bin # P2a: single ffmpeg/ffprobe resolver
23
24
25     @dataclass
26     class HitEvent:
27         t: float # seconds
28         strength: float # onset-envelope peak value (arbitrary units)
29
30
31     def extract_audio(
32         video_path: str, out_wav: Optional[str] = None, sr: int = 16000
33     ) -> str:
34         """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
35         Audio-only decode, so it's fast even for multi-GB video."""
36         if out_wav is None:
37             fd, out_wav = tempfile.mkstemp(suffix=".wav")
38             os.close(fd)
39         cmd = [
40             ffmpeg_bin(),
41             "-y",
42             "-i",
43             video_path,
44             "-vn",
45             "-ac",
46             "1",

```

```

47         "-ar",
48         str(sr),
49         "-c:a",
50         "pcm_s16le",
51         out_wav,
52     ]
53     subprocess.run(
54         cmd, check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
55     )
56     return out_wav
57
58
59 def load_wav(path: str) -> Tuple[np.ndarray, int]:
60     """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
61     with wave.open(path, "rb") as w:
62         sr, n, sw, ch = (
63             w.getframerate(),
64             w.getnframes(),
65             w.getsampwidth(),
66             w.getnchannels(),
67         )
68         raw = w.readframes(n)
69         if sw == 2:
70             a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
71         elif sw == 4:
72             a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
73         elif sw == 1:
74             a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
75         else:
76             raise ValueError(f"unsupported sample width {sw}")
77         if ch > 1:
78             a = a.reshape(-1, ch).mean(axis=1)
79         return a, sr
80
81
82 def _preemphasis(x: np.ndarray, coef: float = 0.97) -> np.ndarray:
83     if len(x) < 2:
84         return x.copy()
85     return np.append(x[0], x[1:] - coef * x[:-1])
86
87
88 def onset_envelope(
89     samples: np.ndarray,
90     sr: int,
91     frame: int = 1024,
92     hop: int = 256,
93     band: Optional[Tuple[float, float]] = None,
94 ) -> Tuple[np.ndarray, float]:
95     """Spectral-flux onset strength per hop (pure-numpy STFT). Returns (env,
96     hop_seconds).
97
98     `band=(lo_hz, hi_hz)` restricts the flux to a frequency band ? a shuttle "thwack"
99     is HF-rich while voices/footsteps live lower, so band-limiting (e.g. 3-9 kHz)
100     suppresses non-shuttle transients. Needs sr high enough for hi_hz (Nyquist =
101     sr/2)."""
102     x = _preemphasis(np.asarray(samples, dtype=np.float32))
103     hop_sec = hop / float(sr)
104     if len(x) < frame:
105         return np.zeros(0, dtype=np.float32), hop_sec
106     win = np.hanning(frame).astype(np.float32)
107     n_frames = 1 + (len(x) - frame) // hop
108     idx = np.arange(frame)[None, :] + hop * np.arange(n_frames)[:, None]
109     frames = x[idx] * win
110     mag = np.abs(np.fft.rfft(frames, axis=1))
111     if band is not None:

```



```
110         freqs = np.fft.rfftfreq(frame, d=1.0 / sr)
```

18. user (2026-07-02T04:47:20.353Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer.

19. user (2026-07-02T04:47:21.020Z)

```
1     """Single resolver for the ffmpeg / ffprobe binaries (PACKAGING_PLAN.md P2a).
2
3     Every ffmpeg/ffprobe invocation in the engine goes through :func:`ffmpeg_bin` /
:func:`ffprobe_bin`
4     so a packaged app can point at a vendored or non-PATH binary via the ``RALLY_FFMPEG`` /
5     ``RALLY_FFPROBE`` env vars, there is ONE place to add a config knob or a louder check
later, and a
6     missing ffmpeg fails with an actionable, OS-specific install hint instead of a cryptic
7     ``FileNotFoundError`` from deep in the pipeline.
8
9     DEFAULT (env unset) returns the bare name (``"ffmpeg"`` / ``"ffprobe"``) ? PATH
resolution, exactly
10    the prior hardcoded literals (byte-identical argv), so this is a pure seam with no
behaviour change.
11    stdlib-only.
12    """
13
14    from __future__ import annotations
15
16    import os
17    import platform
18    import shutil
19
20
21    class FfmpegNotFound(RuntimeError):
22        """Raised by :func:`require_ffmpeg` when ffmpeg/ffprobe can't be located."""
23
24
25    def ffmpeg_bin() -> str:
26        """The ffmpeg executable: ``RALLY_FFMPEG`` if set, else ``"ffmpeg"`` (resolved on
PATH)."""
27        return (os.environ.get("RALLY_FFMPEG") or "").strip() or "ffmpeg"
28
29
30    def ffprobe_bin() -> str:
31        """The ffprobe executable: ``RALLY_FFPROBE`` if set, else ``"ffprobe"`` (resolved on
PATH)."""
32        return (os.environ.get("RALLY_FFPROBE") or "").strip() or "ffprobe"
33
34
35    def ffmpeg_available() -> bool:
36        return shutil.which(ffmpeg_bin()) is not None
37
38
39    def ffprobe_available() -> bool:
40        return shutil.which(ffprobe_bin()) is not None
41
42
43    def install_hint() -> str:
44        """The OS-appropriate one-line install command (mirrors scripts/doctor.py)."""
45        system = platform.system()
46        if system == "Windows":
47            return "winget install Gyan.FFmpeg"
48        if system == "Darwin":
49            return "brew install ffmpeg"
50        return "sudo apt-get install -y ffmpeg (or your distro's package manager)"
```

```

51
52
53     def require_ffmpeg() -> None:
54         """Raise :class:`FfmpegNotFound` (with an OS-specific install hint) if
55         ffmpeg/ffprobe is missing.
56
57         A loud, actionable pre-flight for entry points that are about to shell out ? used by
58         the
59         first-run self-test / wizard; the per-call-site argv still goes through
60         :func:`ffmpeg_bin`. """
61         missing = [
62             name
63             for name, ok in (
64                 ("ffmpeg", ffmpeg_available()),
65                 ("ffprobe", ffprobe_available()),
66             )
67             if not ok
68         ]
69         if missing:
70             raise FfmpegNotFound(
71                 f"{' and '.join(missing)} not found (PATH or RALLY_FFmpeg/RALLY_FFPROBE).
72                 FFmpeg is "
73                 f"required for video processing. Install it: {install_hint()}"
74             )
75

```

20. user (2026-07-02T04:47:27.317Z)

No files found

21. user (2026-07-02T04:47:29.304Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\main.py:1912:
subprocess.run([sys.executable, doctor_path])
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\audio\hit_detector.py:53: subprocess.run(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\stitcher\compiler.py:130: result = subprocess.run(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\stitcher\compiler.py:321: subprocess.run(cmd,
capture_output=True, check=True)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\stitcher\compiler.py:494: subprocess.run(concat_cmd,
capture_output=True, check=True)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\validators\resolution_fps.py:15: result = subprocess.run(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\validators\pts_continuity.py:35: result =
subprocess.run(cmd, capture_output=True, text=True, check=True)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\validators\format.py:43: result = subprocess.run(cmd,
capture_output=True, text=True, check=True)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\worker\__main__.py:382: out = subprocess.check_output(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-

```

```

timeouts\backend\worker\__main__.py:606: rc = subprocess.run(cmd).returncode
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\utils\annotation_proxy.py:31: out = subprocess.check_output(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\utils\annotation_proxy.py:81: subprocess.run(cmd, check=True,
capture_output=True)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\tools\rally_slicer.py:117: out = subprocess.run(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\tools\rally_slicer.py:185: subprocess.run(cmd, capture_output=True,
check=True)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\detectors\base.py:115: return subprocess.run(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\segmenters\gemini.py:166: subprocess.run(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\detectors\native_wasb_runner.py:129: return subprocess.run(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\utils\run_telemetry.py:69: out = subprocess.run(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\utils\run_telemetry.py:123: out = subprocess.run(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\utils\video.py:18: subprocess.check_output(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\utils\video.py:75: subprocess.run(

```

22. user (2026-07-02T04:47:47.665Z)

```

1      """Frame-preserving annotation proxy (STUDIO_MEDIA_LIFECYCLE.md D4).
2
3      A big 4K master is heavy to decode + scrub in a browser. For in-UI annotation we serve a
downscaled
4      proxy that keeps the SAME timeline (fps + duration) ? so a rally marked at t seconds on
the proxy is
5      t seconds on the master, and the labels still key to the master's file MD5. Only
resolution is
6      dropped; duration parity is VERIFIED after the transcode, and a proxy whose timeline
drifted is
7      discarded (the caller falls back to serving the source). Best-effort + stdlib-only;
never raises.
8      """
9      from __future__ import annotations
10
11     import logging
12     import os
13     import subprocess
14     import tempfile
15
16     from backend.utils.ffmpeg import ffmpeg_available, ffmpeg_bin, ffprobe_bin
17     from backend.utils.video import get_video_duration
18
19     logger = logging.getLogger(__name__)
20
21     #: Max proxy height (px). A source already at/below this needs no proxy (serving it is

```

```

already light).
22     DEFAULT_MAX_HEIGHT = 720
23     #: Duration must match the source within this (s), else the timeline drifted ? discard
    (protects the
24     #: second-based marks the annotator produces).
25     _DURATION_TOLERANCE_S = 0.5
26
27
28     def probe_height(path: str) -> int:
29         """The video's height in px, or 0 on failure."""
30         try:
31             out = subprocess.check_output(
32                 [ffmpeg_bin(), "-v", "error", "-select_streams", "v:0", "-show_entries",
33                  "stream=height", "-of", "default=noprint_wrappers=1:nokey=1", path],
34                  text=True,
35                  ).strip().splitlines()
36             return int(out[0]) if out and out[0].strip() else 0
37         except Exception as e: # noqa: BLE001 - probe is best-effort
38             logger.warning("proxy: could not probe height for %s: %s", path, e)
39             return 0
40
41
42     def needs_proxy(src: str, max_height: int = DEFAULT_MAX_HEIGHT) -> bool:
43         """True when ``src`` is tall enough that a downscaled proxy is worth generating."""
44         return probe_height(src) > max_height
45
46
47     def _cleanup(path: str) -> None:
48         try:
49             os.remove(path)
50         except OSError:
51             pass
52
53
54     def make_annotation_proxy(src: str, dst: str, max_height: int = DEFAULT_MAX_HEIGHT) ->
bool:
55         """Transcode ``src`` ? a downscaled, timeline-preserving proxy at ``dst`` (written
atomically).
56         Returns True on a verified proxy, False otherwise (caller serves the source). Never
raises.
57
58         Only the resolution is scaled (``scale=-2:H``); fps/duration are untouched and
duration parity is
59         verified, so second-based marks transfer to the master unchanged. Audio is kept (the
shuttle
60         'thwack' helps the rater judge rally boundaries)."""
61         if not ffmpeg_available():
62             logger.info("proxy: ffmpeg unavailable; serving source (%s)", src)
63             return False
64         if not os.path.isfile(src):
65             return False
66         src_dur = get_video_duration(src)
67         dst_dir = os.path.dirname(dst) or "."
68         os.makedirs(dst_dir, exist_ok=True)
69         fd, tmp = tempfile.mkstemp(dir=dst_dir, prefix=".proxy-", suffix=".mp4")
70         os.close(fd)
71         cmd = [
72             ffmpeg_bin(), "-y", "-i", src,
73             "-vf", f"scale=-2:{max_height}",
74             "-c:v", "libx264", "-preset", "veryfast", "-crf", "28", "-pix_fmt", "yuv420p",
75             "-g", "48", # denser keyframes ? snappy scrubbing
76             "-c:a", "aac", "-b:a", "96k",
77             "-movflags", "+faststart",
78             tmp,
79         ]

```

```

80         try:
81             subprocess.run(cmd, check=True, capture_output=True)
82         except Exception as e: # noqa: BLE001 - a transcode failure is non-fatal; fall back
to the source
83             logger.warning("proxy: ffmpeg failed for %s: %s", src, e)
84             _cleanup(tmp)
85             return False
86         proxy_dur = get_video_duration(tmp)
87         if src_dur > 0 and abs(proxy_dur - src_dur) > _DURATION_TOLERANCE_S:
88             logger.warning("proxy: duration parity off for %s (src=%.3fs proxy=%.3fs);
discarding",
89                             src, src_dur, proxy_dur)
90             _cleanup(tmp)
91             return False
92         try:
93             os.replace(tmp, dst)
94         except OSError as e:
95             logger.warning("proxy: could not place proxy at %s: %s", dst, e)
96             _cleanup(tmp)
97             return False
98         logger.info("proxy: wrote annotation proxy %s (%.1fs)", dst, proxy_dur)
99         return True
100

```

23. user (2026-07-02T04:47:49.059Z)

```

360
361         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
362         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
363         """
364         name = fname.replace(".rallies.csv", "").replace(".csv", "")
365         return _LABEL_DATE_PREFIX.sub("", name)
366
367
368     def _probe_video_fps_width(path: str):
369         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
370
371         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
372         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
373         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
374         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
375         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
376         rather than guesses.
377         """
378         import json
379         import subprocess
380
381         try:
382             out = subprocess.check_output(
383                 [
384                     ffprobe_bin(),
385                     "-v",
386                     "error",
387                     "-select_streams",
388                     "v:0",
389                     "-show_entries",
390                     "stream=r_frame_rate,width",
391                     "-of",

```

```

392         "json",
393         path,
394     ],
395     stderr=subprocess.DEVNULL,
396 ).decode()
397 st = (json.loads(out).get("streams") or [{}])[0]
398 num, den = str(st.get("r_frame_rate", "0/1")).partition("/")
399 fps = float(num) / float(den or "1")
400 width = float(st.get("width") or 0)
401 if fps > 0 and width > 0:
402     return fps, width
403 except (
404     subprocess.SubprocessError,
405     ValueError,
406     KeyError,
407     OSError,
408     json.JSONDecodeError,
409 ):

```

24. user (2026-07-02T04:47:50.957Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\reporting\harvest.py:75: # thwack onset probe (pipeline/audio/hit_detector)
remains a diagnostic / calibration
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\audio\e1_probe.py:9:Reuses: audio.hit_detector,
calibrate_wasb.load_golden_gts/make_predict_fn,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\audio\e1_probe.py:21:from backend.pipeline.audio.hit_detector import
HitEvent, detect_hits_in_video
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\fusion_audio.py:29:from backend.pipeline.audio.hit_detector import
detect_hits_in_video
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\audio\hit_detector.py:31:def extract_audio(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\audio\hit_detector.py:173: wav = extract_audio(video_path, sr=sr)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\detectors\audio_features.py:3:NO I/O ? the audio hit detection lives
in `backend/pipeline/audio/hit_detector.py`

```

25. user (2026-07-02T04:47:58.844Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\audio\e1_probe.py:21:from backend.pipeline.audio.hit_detector import
HitEvent, detect_hits_in_video
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\audio\e1_probe.py:66: hits = detect_hits_in_video(video, k=k)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\fusion_audio.py:4:`detect_hits_in_video` once per video (ffmpeg + numpy,
CPU) and adds the 3 audio features per
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\fusion_audio.py:11: python -m backend.eval.fusion_audio --manifest
output/human_lovo_manifest.json --features-dir output --augment --compare

```

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\fusion_audio.py:29:from backend.pipeline.audio.hit_detector import
detect_hits_in_video
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\fusion_audio.py:44:          f"[fusion_audio] {spec['name']}: no
feature JSON ? run fusion_golden first; skipping"
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\fusion_audio.py:49:          f"[fusion_audio] {spec['name']}: detecting
audio hits (k={k})...",
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\fusion_audio.py:52:          hits = detect_hits_in_video(video, k=k)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\fusion_audio.py:60:          f"[fusion_audio] {spec['name']}:
{len(hits)} hits over {len(data['candidates'])} windows",
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\fusion_audio.py:82:
skip_message="fusion_audio.load_videos_with_audio: skipping %s ? windowing-only "
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\experiment.py:190:    ``fusion_audio.augment`` and older JSONs predate
it).
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\experiment.py:196:    and ``fusion_audio.load_videos_with_audio`` ? SKIP
those rather than crash on ``c["shuttle"]``,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\experiment.py:219:    (``ablation.load_videos`` /
``fusion_compare.load_videos`` / ``fusion_audio.load_videos_with_audio``).
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\detectors\audio_features.py:4:(`detect_hits_in_video` ->
`List[HitEvent]`); this only turns those hit events into a few
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\audio\hit_detector.py:171:def detect_hits_in_video(video_path: str,
sr: int = 16000, **kw) -> List[HitEvent]:

```

26. user (2026-07-02T04:48:00.543Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\worker\__main__.py:375:    probe with ffprobe (same tool as
``get_video_duration``) and return None so the caller SKIPS
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\audio\e1_probe.py:22:from backend.utils.video import get_video_duration
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\audio\e1_probe.py:69:    dur = get_video_duration(video) or (max((e for _,
e in gts + wasb), default=0.0))
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\gemini_refine.py:11:sports.badminton prompt (sport_registry),
utils.video.extract_video_chunk,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\eval\gemini_refine.py:35:from backend.utils.video import extract_video_chunk,
get_video_duration

```

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\eval\gemini_refine.py:93:    if not extract_video_chunk(C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\eval\gemini_refine.py:142:    duration = get_video_duration(video)C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\eval\gemini_refine.py:234:    duration = get_video_duration(video)C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\eval\gemini_refine.py:245:    if not extract_video_chunk(C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\utils\annotation_proxy.py:17:from backend.utils.video import get_video_durationC:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\utils\annotation_proxy.py:66:    src_dur = get_video_duration(src)C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\utils\annotation_proxy.py:86:    proxy_dur = get_video_duration(tmp)C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\stitcher\compiler.py:126:    def _get_video_duration(self, video_path: str) -> float:C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\stitcher\compiler.py:406:        video_duration = self._get_video_duration(raw_video_path)C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\segmenters\gemini.py:17:from backend.utils.video import extract_video_chunk, get_video_durationC:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\segmenters\gemini.py:232:        success = extract_video_chunk(C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\segmenters\gemini.py:636:        video_duration = get_video_duration(video_to_upload)C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\segmenters\trajectory_hybrid.py:35:from backend.utils.video import extract_video_chunk, get_video_durationC:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\segmenters\trajectory_hybrid.py:222:        video_duration = get_video_duration(video_path)C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\segmenters\trajectory_hybrid.py:395:        if not extract_video_chunk(C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\segmenters\yolo_hybrid.py:30:from backend.utils.video import extract_video_chunk, get_video_durationC:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\segmenters\yolo_hybrid.py:155:        success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\pipeline\segmenters\yolo_hybrid.py:205:        success = extract_video_chunk(C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
```



```

indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\segmenters\yolo_hybrid.py:341:          success =
extract_video_chunk(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\segmenters\yolo_hybrid.py:477:          video_duration =
get_video_duration(video_path)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\utils\video.py:13:def get_video_duration(video_path: str) -> float:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\utils\video.py:39:def extract_video_chunk(

```

27. user (2026-07-02T04:49:09.064Z)

```

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\main.py:1132:def _execute_processing(
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\main.py:1644:def _execute_compile(cfg, db, job: Dict[str, Any], progress=None)
-> Dict[str, Any]:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\pipeline\validators\base.py:50:    def run_all_with_context(self, video_path:
str, config: Any):

```

28. user (2026-07-02T04:49:14.615Z)

```

1644     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1645         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1646         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1647         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1648         so the SPA always gets a clean verdict instead of a worker-crash error."""
1649         notify = progress or (lambda stage: None)
1650         params = json.loads(job.get("params") or "{}")
1651         video_id = job.get("video_id") or ""
1652         segment_ids = params.get("segment_ids") or []
1653         locale = params.get("locale")
1654
1655         notify("resolving")
1656         try:
1657             video, kept, out_name = _resolve_compile(
1658                 db, cfg, video_id, segment_ids, params.get("output_filename", "")
1659             )
1660         except _CompileError as e:
1661             return {"status": "failed", "message": e.detail, "video_id": video_id}
1662
1663         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1664         output_dir = (
1665             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1666         )
1667         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1668         # unknown/absent locale keeps the configured default unchanged.
1669         localized_watermark = localize_watermark(stitching.watermark, locale)
1670         compiler = VideoCompiler(
1671             output_dir,
1672             begin_padding=stitching.begin_padding,
1673             end_padding=stitching.end_padding,
1674             watermark=localized_watermark,
1675         )

```

```

1676         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1677
1678         notify("stitching")
1679         try:
1680             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
1681             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
1682             # same seam the process path uses); without it, compiling a bucket-ingested
1683             # video can't open gs://.
1684             local_src = storage.acquire_local_input(
1685                 video["filepath"], getattr(cfg, "storage", None)
1686             )
1687             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1688         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
1689             FAILURE, not a crash
1690             logger.error(
1691                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1692             )
1693             return {
1694                 "status": "failed",
1695                 "message": f"Highlight stitching failed: {e}",
1696                 "video_id": video_id,
1697             }
1698
1699         # `filepath` is server-local (not browser-reachable); also return the download URL
1700         # so the SPA needn't
1701         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1702         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1703         notify("finished")
1704         return {
1705             "status": "success",
1706             "filepath": out_path,
1707             "download_url": download_url,
1708             "video_id": video_id,
1709         }
1710
1711     # --- CLI Debug Utility & Orchestration ---
1712     def run_cli_diagnose_clip(video_path: str, timestamp: float):
1713         """CLI debugging utility to examine segment properties around a timestamp."""
1714         print("=" * 60)
1715         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
1716         print("=" * 60)
1717
1718         cfg = load_config() # legacy wrapper still works, returns dict for now
1719
1720         # 1. Validation check diagnostics
1721         print("[DIAGNOSTIC] Running validation framework...")
1722         results = registry.run_all(video_path, cfg)
1723         for r in results:
1724             status_symbol = "[PASS]" if r.passed else "[FAIL]"
1725             print(f"    {status_symbol} {r.validator_name}: {r.message}")
1726             if r.details:
1727                 print(f"        Details: {r.details}")
1728
1729         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
1730         print("-" * 60)
1731         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
1732         # Lazy import: cv2 is NOT installed in the LIGHT tier (torch-free default)
1733         # and this --debug-clip path is CLI-only, not on the served API path.
1734         import cv2

```

```

30         ) -> ValidationResult:
31             """
32             Executes validation.
33             - video_path: Local path to the MP4 file.
34             - config: AppConfig (typed) ? callers pass the typed model; attribute access
35               preferred over .get() dict chains.
36             - context: Dynamic context dict shared across validators (stores shared values
37               like ffprobe metadata).
38             """
39             pass
40
41
42     class ValidationRegistry:
43         def __init__(self):
44             self._validators: List[VideoValidator] = []
45
46         def register(self, validator: VideoValidator):
47             """Registers a new validator."""
48             self._validators.append(validator)
49
50         def run_all_with_context(self, video_path: str, config: Any):
51             """Runs all validators and returns ``(results, context)``.
52
53             The per-run ``context`` accumulates shared values (notably
54             ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
55             discards it; callers that want media metadata for an observability report
56             use this variant instead of re-probing.
57             """
58             results = []
59             context: dict[
60                 str, Any
61             ] = {} # Shared context (e.g. for caching ffprobe outputs)
62             for val in self._validators:
63                 try:
64                     res = val.validate(video_path, config, context)
65                     results.append(res)
66                 except Exception as e:
67                     results.append(
68                         ValidationResult(
69                             validator_name=val.name,
70                             passed=False,
71                             message=f"Validator threw an unexpected error: {str(e)}",
72                         )
73                     )
74             return results, context
75
76         def run_all(self, video_path: str, config: Any) -> List[ValidationResult]:
77             """Runs all registered validators in sequence, accumulating results."""
78             results, _ = self.run_all_with_context(video_path, config)
79             return results
80
81
82     # Shared global registry
83     registry = ValidationRegistry()
84

```

30. user (2026-07-02T04:49:28.557Z)

```

1132     def _execute_processing(
1133         config=None, db=None, req=None, progress=None
1134     ) -> Dict[str, Any]:
1135         """The full hash ? validate ? segment ? report pipeline for one video.
1136
1137         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
1138         module-level ``global_config``/``db_instance`` (backward-compatible with existing

```

```

1139     tests that pass only ``req``). Route handlers always pass explicit params; older
1140     test code that calls ``_execute_processing(req)`` still works.
1141
1142     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
1143     Raises on unexpected internal errors ? the caller records those as a job failure
1144     """
1145     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
1146     # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
1147     if config is not None and not isinstance(config, AppConfig):
1148         # Old signature: _execute_processing(req, progress=None)
1149         if req is None and db is not None and isinstance(db, ProcessRequest):
1150             req = db
1151             db = None
1152         elif req is None and isinstance(config, ProcessRequest):
1153             req = config
1154             config = None
1155     # Resolve config/db from module globals when not passed explicitly
1156     if config is None:
1157         config = global_config
1158     if db is None:
1159         db = db_instance
1160
1161     notify = progress or (lambda stage: None)
1162
1163     notify("hashing")
1164     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
1165     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
1166     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
1167     # consumers (hash / validators / segmenter) use the resolved local path.
1168     local_video = storage.acquire_local_input(
1169         req.video_path, getattr(config, "storage", None)
1170     )
1171     video_id = compute_video_id(local_video)
1172     was_reingest = db.get_video(video_id) is not None
1173
1174     # 1. Run pluggable validators (with-context variant surfaces ffmpegprobe_meta for the
report)
1175     notify("validating")
1176     logger.info(f"Running validations for {req.video_path}...")
1177     val_results, val_context = registry.run_all_with_context(local_video, config)
1178     failures = [r for r in val_results if not r.passed]
1179
1180     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
1181     default_seg = getattr(
1182         getattr(config, "indexing", None), "default_segmenter", "motion"
1183     )
1184     seg_name = req.segmenter or default_seg
1185     segmenter_actual = None
1186     segmentation_ran = False
1187     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
1188     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
1189     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
1190     compute_actual: Optional[str] = None
1191     response: Optional[Dict[str, Any]] = None
1192     try:
1193         if failures:
1194             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates

```

```
1195         # status; it no longer destroys the video's prior good segments.
1196         db.add_video(
1197             video_id,
1198             req.video_path,
1199             "failed",
1200             [r.model_dump() for r in val_results],
1201         )
1202         response = {
1203             "video_id": video_id,
1204             "status": "failed",
1205             "message": "Video failed validation checks.",
1206             "results": [r.model_dump() for r in val_results],
1207         }
1208         return response
1209
1210     # 2. Run segmenter & indexer
1211     logger.info("[API] Video passed checks. Segmenting and indexing...")
```

31. user (2026-07-02T04:49:29.577Z)

No files found

32. user (2026-07-02T04:49:36.175Z)

```
__init__.py
test_ablation.py
test_annotation_providers.py
test_annotation_proxy.py
test_api_annotations.py
test_api_assets.py
test_api_cloud_dispatch.py
test_api_endpoints.py
test_app_home.py
test_app_launcher.py
test_async_jobs.py
test_audio_features.py
test_automated_provider.py
test_base.py
test_billing.py
test_bind_and_host.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_check_md_links.py
test_cleanup_caches.py
test_compiler.py
test_compute_backend.py
test_config.py
test_config_deployment.py
test_config_wasb.py
test_court_gate.py
test_database.py
test_db_upgrade_safety.py
test_detector_keying.py
test_device_context.py
test_distill_local.py
test_doctor.py
test_edge_auth.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
```

test_eval_stats.py
test_eval_training.py
test_execution_policy.py
test_experiment.py
test_experiment_honest.py
test_export_wasb_segments.py
test_ffmpeg_resolver.py
test_flywheel.py
test_freespace.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
test_gemini.py
test_gemini_refine.py
test_generate_highlights.py
test_geometry.py
test_golden_fixtures.py
test_golden_fixtures_split.py
test_golden_real_fixtures.py
test_gt_loader.py
test_hashing.py
test_hit_detector.py
test_import_smoke.py
test_input_source.py
test_job_self_scheduling.py
test_job_worker_loop.py
test_job_worker_module_seam.py
test_leak_scan.py
test_light_tier_no_torch.py
test_locale_analytics.py
test_logging_setup.py
test_motion_segmenter.py
test_native_wasb_runner.py
test_nightly_reelcount.py
test_nightly_regression.py
test_output_base.py
test_p2b_install.py
test_path_safety.py

33. user (2026-07-02T04:49:43.389Z)

test_reingest_safety.py
test_reliability_hardening.py
test_reporting.py
test_reporting_harvest.py
test_reporting_spec_faq.py
test_results.py
test_run_signals.py
test_run_telemetry.py
test_score_calibration.py
test_segmentation_metrics.py
test_serve_contrast.py
test_served_gate_a_regression.py
test_server_entrypoint.py
test_single_instance.py
test_splits.py
test_sport_profiles.py
test_startup_checks.py
test_storage.py
test_storage_capabilities.py
test_storage_gdrive_api.py
test_stub_runner.py
test_tolerance_metrics.py
test_tracknet_hybrid.py

```
test_tracknet_runner.py
test_training_gen0.py
test_trajectory_hybrid_equivalence.py
test_ui_bundle.py
test_upload_download.py
test_validators_base.py
test_variant_tuning.py
test_video.py
test_wasb_cache.py
test_wasb_runner.py
test_watermark.py
test_watermark_i18n.py
test_windowing.py
test_worker.py
test_workspace.py
test_wsl_tilde_quoting.py
test_yolo_hybrid.py
```

34. user (2026-07-02T04:49:44.943Z)

```
5:import subprocess
59:  with patch.object(subprocess, "run", side_effect=fake_run):
82:  with patch.object(subprocess, "run", side_effect=fake_run):
```

35. user (2026-07-02T04:49:50.719Z)

```
1      """VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
2      where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""
3
4      import os
5      import subprocess
6      from unittest.mock import MagicMock, patch
7
8      import pytest
9
10     from backend.pipeline.stitcher.compiler import VideoCompiler
11
12
13     def test_parse_time_formats():
14         f = VideoCompiler._parse_time
15         assert f(12.5) == 12.5
16         assert f("12.5") == 12.5
17         assert f("1:30") == 90.0
18         assert f("1:00:00") == 3600.0
19         assert f("garbage") == 0.0
20
21
22     def test_merge_overlapping_collapses_duplicates_and_overlaps():
23         m = VideoCompiler._merge_overlapping
24         # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse
25         # to one each.
26         assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [
27             (28.2, 34.5),
28             (40.5, 45.5),
29         ]
30         # Unsorted input is sorted; an overlapping range extends the previous.
31         assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [
32             (0.0, 5.0),
33             (100.0, 120.0),
34         ]
35         # Genuinely distinct (gapped) rallies are preserved, not merged.
36         assert m([(0.0, 5.0), (6.0, 9.0)]) == [(0.0, 5.0), (6.0, 9.0)]
37         # Degenerate rows (end <= start) are dropped; string timestamps are accepted.
38         assert m([("0:10", "0:20"), (30.0, 30.0), (31.0, 29.0)]) == [(10.0, 20.0)]
```

```

39
40 def test_compile_dedups_before_stitching(tmp_path):
41     """A segment list with every rally twice must yield ONE clip per unique rally."""
42     comp = VideoCompiler(str(tmp_path / "out"))
43     raw = tmp_path / "raw.mp4"
44     raw.write_bytes(b"x")
45     trims = []
46
47     def fake_run(cmd, *a, **k):
48         if cmd[0] == "ffprobe":
49             return MagicMock(stdout="100.0\n", returncode=0)
50         out = cmd[-1]
51         if isinstance(out, str) and out.endswith(".mp4"):
52             open(out, "wb").write(b"clip")
53             # count per-clip trims (the concat list file input is not an .mp4 output)
54             if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
55                 trims.append(cmd)
56             return MagicMock(returncode=0, stdout=b"", stderr=b"")
57
58     dup_segments = [(0.0, 5.0), (0.2, 5.0), (10.0, 15.0), (10.0, 15.0), (20.0, 25.0)]
59     with patch.object(subprocess, "run", side_effect=fake_run):
60         comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
61         # 5 input segments collapse to 3 unique rallies -> 3 per-clip trims, not 5.
62         assert len(trims) == 3, f"expected 3 deduped trims, got {len(trims)}"
63
64
65 def test_compile_basenames_output_and_stitches(tmp_path):
66     comp = VideoCompiler(str(tmp_path / "out"))
67     raw = tmp_path / "raw.mp4"
68     raw.write_bytes(b"x")
69
70     calls = []
71
72     def fake_run(cmd, *a, **k):
73         calls.append(cmd)
74         # Create whatever output file ffmpeg/ffprobe was told to write so existence
75         if cmd[0] == "ffprobe":
76             return MagicMock(stdout="100.0\n", returncode=0)
77         out = cmd[-1]
78         if isinstance(out, str) and out.endswith(("mp4",)):
79             open(out, "wb").write(b"clip")
80             return MagicMock(returncode=0, stdout=b"", stderr=b"")
81
82     with patch.object(subprocess, "run", side_effect=fake_run):
83         # Traversal in the name must be stripped (defense-in-depth basename).
84         out_path = comp.compile_highlights(
85             str(raw), [(0.0, 5.0), (10.0, 15.0)], "../reel.mp4"
86         )
87
88     assert os.path.basename(out_path) == "reel.mp4"
89     assert os.path.dirname(out_path).endswith("out")
90     # Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
91     assert sum(1 for c in calls if c[0] == "ffmpeg") >= 3
92
93
94 def test_compile_raises_on_no_segments(tmp_path):
95     comp = VideoCompiler(str(tmp_path / "out"))
96     raw = tmp_path / "raw.mp4"
97     raw.write_bytes(b"x")
98     with pytest.raises(ValueError):
99         comp.compile_highlights(str(raw), [], "reel.mp4")
100
101
102 def test_compile_raises_when_source_missing(tmp_path):

```



```
103         comp = VideoCompiler(str(tmp_path / "out"))
104         with pytest.raises(FileNotFoundError):
105             comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
106
```

36. user (2026-07-02T04:50:00.118Z)

```
1     from unittest.mock import patch
2
3     from backend.utils.video import extract_video_chunk, get_video_duration
4
5
6     @patch("subprocess.check_output")
7     def test_get_video_duration_success(mock_check_output):
8         mock_check_output.return_value = b"123.45\n"
9         dur = get_video_duration("dummy.mp4")
10        assert dur == 123.45
11
12
13    @patch("subprocess.check_output")
14    def test_get_video_duration_failure(mock_check_output):
15        mock_check_output.side_effect = Exception("ffprobe error")
16        dur = get_video_duration("dummy.mp4")
17        assert dur == 0.0
18
19
20    @patch("subprocess.run")
21    def test_extract_video_chunk_success(mock_run):
22        success = extract_video_chunk("in.mp4", 10.0, 5.0, "out.mp4")
23        assert success is True
24        mock_run.assert_called_once()
25
26
27    @patch("subprocess.run")
28    def test_extract_video_chunk_failure(mock_run):
29        mock_run.side_effect = Exception("ffmpeg error")
30        success = extract_video_chunk("in.mp4", 10.0, 5.0, "out.mp4")
31        assert success is False
32
```

37. user (2026-07-02T04:50:00.378Z)

No files found

38. user (2026-07-02T04:50:10.762Z)

Found 2 files

C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\tests\test_reporting_harvest.py
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\tests\test_api_endpoints.py

39. user (2026-07-02T04:50:11.574Z)

```
1     """Tests for the validation registry, focused on the backward-compatible
2     `run_all_with_context` added for observability reporting."""
3
4     from backend.config.models import AppConfig
5     from backend.pipeline.validators.base import (
6         ValidationRegistry,
7         ValidationResult,
8         VideoValidator,
```

```

9         )
10
11
12     class _ProducerValidator(VideoValidator):
13         @property
14         def name(self):
15             return "producer"
16
17         @property
18         def description(self):
19             return "writes ffprobe_meta into the shared context"
20
21         def validate(self, video_path, config, context):
22             context["ffprobe_meta"] = {"fps": 59.94, "width": 1920, "height": 1080}
23             return ValidationResult(validator_name=self.name, passed=True, message="ok")
24
25
26     class _ConsumerValidator(VideoValidator):
27         @property
28         def name(self):
29             return "consumer"
30
31         @property
32         def description(self):
33             return "reads ffprobe_meta the producer cached"
34
35         def validate(self, video_path, config, context):
36             seen = "ffprobe_meta" in context
37             return ValidationResult(
38                 validator_name=self.name, passed=seen, message=f"saw_meta={seen}"
39             )
40
41
42     def _registry():
43         r = ValidationRegistry()
44         r.register(_ProducerValidator())
45         r.register(_ConsumerValidator())
46         return r
47
48
49     def test_run_all_with_context_returns_results_and_context():
50         results, context = _registry().run_all_with_context("v.mp4", AppConfig())
51         assert [r.validator_name for r in results] == ["producer", "consumer"]
52         assert context["ffprobe_meta"]["fps"] == 59.94
53         assert results[1].passed is True # consumer saw the shared context
54
55
56     def test_run_all_is_unchanged_and_returns_only_results():
57         results = _registry().run_all("v.mp4", AppConfig())
58         assert isinstance(results, list)
59         assert all(isinstance(r, ValidationResult) for r in results)
60         assert [r.validator_name for r in results] == ["producer", "consumer"]

```

40. user (2026-07-02T04:51:10.977Z)

```

1     """Unit tests for the frame-preserving annotation proxy (D4) ? ffmpeg is MOCKED so these
run offline:
2     the transcode command is stubbed and duration parity is fed in, exercising the success /
parity-drift
3     / no-ffmpeg branches without a real encode."""
4     import os
5
6     import backend.utils.annotation_proxy as ap
7
8

```

```

9     def _stub_ffmpeg(monkeypatch, writes=b"proxy"):
10         monkeypatch.setattr(ap, "ffmpeg_available", lambda: True)
11
12         def fake_run(cmd, check, capture_output):
13             with open(cmd[-1], "wb") as f: # the util passes the temp output as the last
14                 f.write(writes)
15
16             class _R: # a minimal CompletedProcess stand-in
17                 returncode = 0
18
19             return _R()
20
21         monkeypatch.setattr(ap.subprocess, "run", fake_run)
22
23
24     def test_make_proxy_success_when_duration_matches(tmp_path, monkeypatch):
25         _stub_ffmpeg(monkeypatch, b"proxy-bytes")
26         monkeypatch.setattr(ap, "get_video_duration", lambda p: 12.0) # src + proxy both
12.0 ? parity ok
27         src = tmp_path / "src.mp4"
28         src.write_bytes(b"x" * 10)
29         dst = tmp_path / "proxies" / "v.mp4"
30         assert ap.make_annotation_proxy(str(src), str(dst)) is True
31         assert dst.read_bytes() == b"proxy-bytes"
32
33
34     def test_make_proxy_discards_on_duration_drift(tmp_path, monkeypatch):
35         _stub_ffmpeg(monkeypatch)
36         durs = iter([12.0, 20.0]) # src=12, proxy=20 ? timeline drifted ? discard (protect
timestamps)
37         monkeypatch.setattr(ap, "get_video_duration", lambda p: next(durs))
38         src = tmp_path / "src.mp4"
39         src.write_bytes(b"x" * 10)
40         dst = tmp_path / "proxies" / "v.mp4"
41         assert ap.make_annotation_proxy(str(src), str(dst)) is False
42         assert not dst.exists()
43         assert [f for f in os.listdir(dst.parent) if f.startswith(".proxy-")] == [] # temp
cleaned up
44
45
46     def test_make_proxy_false_without_ffmpeg(tmp_path, monkeypatch):
47         monkeypatch.setattr(ap, "ffmpeg_available", lambda: False)
48         src = tmp_path / "src.mp4"
49         src.write_bytes(b"x")
50         assert ap.make_annotation_proxy(str(src), str(tmp_path / "p.mp4")) is False
51
52
53     def test_probe_height_and_needs_proxy(monkeypatch):
54         monkeypatch.setattr(ap.subprocess, "check_output", lambda *a, **k: "1080\n")
55         assert ap.probe_height("x.mp4") == 1080
56         assert ap.needs_proxy("x.mp4", max_height=720) is True # 1080 > 720 ? worth a
proxy
57         assert ap.needs_proxy("x.mp4", max_height=1080) is False # equal ? not worth it
58
59
60     def test_probe_height_zero_on_failure(monkeypatch):
61         def boom(*a, **k):
62             raise OSError("no ffprobe")
63         monkeypatch.setattr(ap.subprocess, "check_output", boom)
64         assert ap.probe_height("x.mp4") == 0
65         assert ap.needs_proxy("x.mp4") is False # unknown height ? don't attempt a proxy
66

```

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\tests\test_config_wasb.py:55: assert "debug_duration" in fields
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\tests\test_config_wasb.py:57: cfg = AppConfig(indexing={"yolo_prefetch_workers": 7, "debug_duration": 30})
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\tests\test_config_wasb.py:60: assert idx.debug_duration == 30
```

42. user (2026-07-02T04:51:24.219Z)

```
1 import logging
2 import os
3 from unittest.mock import MagicMock, patch
4
5 from backend.config.models import AppConfig
6 from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter, _chunk_workdir
7 from backend.reporting import run_signals
8 from backend.results import Failure
9
10
11 def test_chunk_workdir_is_per_video():
12     """Concurrency isolation: two videos must get DIFFERENT chunk workdirs under
13     output/chunks, so one run finishing (and rmdir-ing its dir) can't undo another
14     run's in-flight chunks. Regression for the shared-output/chunks clobber."""
15     a, b = _chunk_workdir("vidA"), _chunk_workdir("vidB")
16     assert a != b
17     assert a == os.path.join("output", "chunks", "vidA")
18     assert os.path.dirname(a) == os.path.dirname(b) == os.path.join("output", "chunks")
19
20
21 @patch("backend.pipeline.segmenters.gemini.provider_registry")
22 @patch("backend.pipeline.segmenters.gemini.get_video_duration")
23 @patch("os.environ.get")
24 def test_gemini_segmenter_single_file(
25     mock_env_get, mock_get_dur, mock_provider_registry
26 ):
27     mock_env_get.return_value = "dummy_api_key"
28     mock_get_dur.return_value = 100.0 # 100s video
29
30     mock_provider = MagicMock()
31     mock_provider.analyze_video.return_value = (
32         [{"start_time": 10.0, "end_time": 20.0, "shuttle_exchange_count": 5}],
33         {"upload_time": 1, "process_time": 1, "gen_time": 1},
34     )
35     mock_provider_registry.get.return_value = mock_provider
36
37     db_mock = MagicMock()
38     db_mock.add_play_segment.return_value = "segment_1"
39
40     config = {
41         "indexing": {
42             "chunk_duration": 300, # Large chunk duration, falls back to single file
43             "chunk_overlap": 30,
44         }
45     }
46
47     segmenter = GeminiPlaySegmenter(db_mock, config)
48     res = segmenter.process_video("v1", "dummy.mp4")
49
50     assert len(res) == 1
51     assert res[0]["id"] == "segment_1"
52     assert res[0]["start_time"] == 10.0
```

```

53         assert res[0]["end_time"] == 20.0
54         assert res[0]["metadata"]["shot_count"] == 5
55
56
57         @patch("backend.pipeline.segmenters.gemini.provider_registry")
58         @patch("backend.pipeline.segmenters.gemini.get_video_duration")
59         @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
60         @patch("os.environ.get")
61         def test_gemini_segmenter_chunked(
62             mock_env_get, mock_extract, mock_get_dur, mock_provider_registry
63         ):
64             mock_env_get.return_value = "dummy_api_key"
65             mock_get_dur.return_value = 200.0 # 200s video
66             mock_extract.return_value = True
67
68             mock_provider = MagicMock()
69
70             # Chunk 1 returns 1 segment, Chunk 2 returns another
71             def analyze_video_side_effect(path, prompt, chunk_duration, label, **kwargs):
72                 if "Chunk 1" in label:
73                     return [{"start_time": 10.0, "end_time": 20.0}], {}
74                 else:
75                     return [{"start_time": 30.0, "end_time": 40.0}], {}
76
77             mock_provider.analyze_video.side_effect = analyze_video_side_effect
78             mock_provider_registry.get.return_value = mock_provider
79
80             db_mock = MagicMock()

```

43. user (2026-07-02T04:51:28.378Z)

CLAUDE.md
 README.md
 backend
 config.json
 deploy
 docs
 eval_baselines
 mypy.ini
 pyproject.toml
 pytest.ini
 requirements.txt
 ruff.toml
 run_all_tests.py
 run_eval.py
 run_regression.py
 scratch
 scripts
 tests
 tools
 training

Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would bury blame history for zero behavior value).

Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle warnings (W) + bugbear (B) + isort (I ? consistent import ordering).

target-version = "py38"

scratch/ = throwaway debugging scripts that may hit real APIs; it is also excluded from pytest collection (see pytest.ini).

```
extend-exclude = ["scratch"]
```

```
[lint]
extend-select = ["W", "B", "I"]
```

```
[lint.per-file-ignores]
```

Entrypoint scripts deliberately run setup (load_dotenv, logging config, sys.path) before importing backend modules ? import order is load-bearing.

```
"backend/main.py" = ["E402"]
"scripts/run_phase3_eval.py" = ["E402"]
"scripts/run_process_and_stitch.py" = ["E402"]
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/mypy.ini
4:# (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
54:     "mypy",
```

44. user (2026-07-02T04:51:36.327Z)

No matches found

45. user (2026-07-02T04:51:37.323Z)

```
1      [mypy]
2      # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3      # modules and pins an explicit quarantine list below ? not a typing crusade.
4      # Ratchet: when you touch a quarantined module, try removing its entry.
5      ignore_missing_imports = True
6      # backend/ is a namespace package (no root __init__.py) ? required so mypy
7      # maps files to backend.* module names instead of duplicating top-level names.
8      explicit_package_bases = True
9
10     # --- Quarantine (known-loose modules, each with a reason) ---
11
12     # Optional module globals (db_instance/global_config set by create_app() / main()).
13     # Route handlers use request.app.state exclusively; helpers take explicit params.
14     # backend.api.job_worker lifted from quarantine (now takes explicit config/db params).
15     [mypy-backend.main]
16     ignore_errors = True
17
18     # Worker dispatcher: composes the SAME validator/segmenter/report building blocks as
19     # backend.main, which are typed `config: dict` but receive the AppConfig migration shim
20     # at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine
21     # until the typed-config migration removes the dict?AppConfig duality everywhere.
22     [mypy-backend.worker.__main__]
23     ignore_errors = True
24
25     # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
26     # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
27     # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
28     # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...])
29     # annotation) ? ratchet those out in a focused typing pass, not here.
30     # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
31     [mypy-backend.pipeline.segmenters.yolo_hybrid]
32     ignore_errors = True
33
34     [mypy-backend.pipeline.segmenters.gemini]
35     ignore_errors = True
36
37     # Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
38     [mypy-backend.pipeline.segmenters.base]
39     ignore_errors = True
40
```

46. user (2026-07-02T04:52:28.236Z)

```
1      import concurrent.futures
2      import logging
3      import os
4      import subprocess
5      import time
6      from typing import Any, Callable, Dict, List, Tuple
7
8      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
9      from backend.providers import provider_registry
10     from backend.reporting import run_signals
11     from backend.results import (
12         Failure, # Finding B: an unrunnable config (missing key/unknown sport/provider) ? a
loud Failure, not a silent 0-rally "success"
13     )
14     from backend.sports import sport_registry
15     from backend.utils.ffmpeg import ffmpeg_bin # P2a: single ffmpeg/ffprobe resolver
16     from backend.utils.paths import output_base
17     from backend.utils.video import extract_video_chunk, get_video_duration
18
19     logger = logging.getLogger(__name__)
20
21
22     def _chunk_workdir(video_id: str, base: str = "output") -> str:
23         """Per-video chunk staging dir under <base>/chunks/<video_id> (`base` default
24         ``output``
25         keeps the prior literal byte-identical; the caller passes the configured output base
26         staging follows --output-dir ? COMPUTE_DECOUPLED_SERVING M3).
27
28         Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
29         otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
30         in-flight work. A per-video subdir means each run only ever creates/cleans its own.
31         """
32         return os.path.join(base, "chunks", video_id)
33
34     def _parse_time(val) -> float:
35         """Converts a timestamp value to total float seconds.
36         Handles float/int pass-through, plain numeric strings, and
37         colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
38         when unexpected formats are encountered."""
39         if isinstance(val, (int, float)):
40             return float(val)
41         if isinstance(val, str):
42             val = val.strip()
43             if ":" in val:
44                 # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
45                 # Convert it, but warn so the issue is visible in logs.
46                 parts = val.split(":")
47                 parts.reverse()
48                 total = 0.0
49                 for i, p in enumerate(parts):
50                     try:
51                         total += float(p) * (60**i)
52                     except ValueError:
53                         pass
54                 logger.warning(
55                     f"Timestamp '{val}' was in colon-separated format (MM:SS/HH:MM:SS)
instead of decimal seconds. Converted to {total:.2f}s."
56                 )
57             return total
58         try:
59             return float(val)
```

```

60         except ValueError:
61             logger.warning(
62                 f"Could not parse timestamp string '{val}' as a number. Defaulting to
0.0."
63             )
64             return 0.0
65         logger.warning(
66             f"Unexpected timestamp type {type(val).__name__} for value '{val}'. Defaulting
to 0.0."
67         )
68         return 0.0
69
70
71     class GeminiPlaySegmenter(BasePlaySegmenter):
72         @property
73         def name(self) -> str:
74             return "gemini"
75
76         @property
77         def description(self) -> str:
78             return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
79
80         def _resolve_provider(self, model_name: str):
81             """Resolve the sport profile, provider, and API key (the process_video
preamble).
82
83             Returns (sport_profile, provider) on success, or a ``Failure`` (with a user-
facing
84             message) when the run cannot proceed ? an unknown sport, a MISSING API KEY, or
an
85             unknown provider. The caller propagates the Failure so the job is marked
**failed**
86             with a reason, instead of a silent 0-rally "success" that looks like an empty
match
87             (Finding B ? the loud-failures bar). A genuine "ran, found nothing" is
unaffected.
88             """
89             # Get sport profile config (config is always typed AppConfig ?
_normalize_config()
90             # in base.py converts plain dicts passed by tests at init time)
91             sport_name = self.config.sport
92             try:
93                 sport_profile = sport_registry.get(sport_name)
94             except KeyError as e:
95                 logger.error(str(e))
96                 return Failure(str(e))
97
98             # Get AI provider and model config
99             provider_name = self.config.ai_provider
100

```

47. user (2026-07-02T04:52:29.395Z)

```

1     """The single ffmpeg/ffprobe resolver (PACKAGING_PLAN.md P2a)."""
2
3     import subprocess
4
5     import pytest
6
7     from backend.utils import ffmpeg as ff
8
9
10    @pytest.fixture(autouse=True)
11    def _clear_ffmpeg_env(monkeypatch):

```



```

12     monkeypatch.delenv("RALLY_FFMPEG", raising=False)
13     monkeypatch.delenv("RALLY_FFPROBE", raising=False)
14
15
16 # ---- default (env unset) is byte-identical to the old hardcoded literals ----
17
18
19 def test_default_bins_are_bare_names():
20     assert ff.ffmpeg_bin() == "ffmpeg"
21     assert ff.ffprobe_bin() == "ffprobe"
22
23
24 def test_env_override(monkeypatch):
25     monkeypatch.setenv("RALLY_FFMPEG", "/opt/ff/ffmpeg")
26     monkeypatch.setenv("RALLY_FFPROBE", "/opt/ff/ffprobe")
27     assert ff.ffmpeg_bin() == "/opt/ff/ffmpeg"
28     assert ff.ffprobe_bin() == "/opt/ff/ffprobe"
29
30
31 def test_blank_env_falls_back_to_default(monkeypatch):
32     monkeypatch.setenv("RALLY_FFMPEG", " ")
33     assert ff.ffmpeg_bin() == "ffmpeg"
34
35
36 # ---- availability + require ----
37
38
39 def test_available_reflects_which(monkeypatch):
40     monkeypatch.setattr(ff.shutil, "which", lambda name: "/usr/bin/" + name)
41     assert ff.ffmpeg_available() is True and ff.ffprobe_available() is True
42     monkeypatch.setattr(ff.shutil, "which", lambda name: None)
43     assert ff.ffmpeg_available() is False and ff.ffprobe_available() is False
44
45
46 def test_require_ffmpeg_noop_when_present(monkeypatch):
47     monkeypatch.setattr(ff.shutil, "which", lambda name: "/usr/bin/" + name)
48     ff.require_ffmpeg() # must not raise
49
50
51 def test_require_ffmpeg_raises_with_hint_when_missing(monkeypatch):
52     monkeypatch.setattr(ff.shutil, "which", lambda name: None)
53     with pytest.raises(ff.FfmpegNotFound) as exc:
54         ff.require_ffmpeg()
55     msg = str(exc.value)
56     assert "ffmpeg" in msg and "ffprobe" in msg
57     assert ff.install_hint() in msg # actionable, OS-specific
58
59
60 @pytest.mark.parametrize(
61     "system, expect", [("Windows", "winget"), ("Darwin", "brew"), ("Linux", "apt-get")]
62 )
63 def test_install_hint_per_os(monkeypatch, system, expect):
64     monkeypatch.setattr(ff.platform, "system", lambda: system)
65     assert expect in ff.install_hint()
66
67
68 # ---- a wrapped call site actually uses the resolved binary ----
69
70
71 def test_call_site_uses_resolved_binary(monkeypatch):
72     from backend.utils import video
73
74     monkeypatch.setenv("RALLY_FFPROBE", "/custom/ffprobe")
75     captured = {}
76

```

```

77     def _fake_check_output(argv, *a, **k):
78         captured["argv"] = argv
79         return b"12.5"
80
81     monkeypatch.setattr(subprocess, "check_output", _fake_check_output)
82     assert video.get_video_duration("x.mp4") == 12.5
83     assert (
84         captured["argv"][0] == "/custom/ffprobe"
85     ) # the override flows through to argv[0]
86

```

48. user (2026-07-02T04:52:38.525Z)

```

1     """Path-safety helpers for the API boundary (untrusted request fields)."""
2
3     import os
4     from typing import Any, Optional
5
6
7     def output_base(config: Any) -> str:
8         """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
9         (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
10        so the
11        chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
12        ``output.output_dir`` instead of
13        a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
14        literal.
15        This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
16        may be a
17        ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
18        dict-shim
19        (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
20        ``"output"``. """
21        try:
22            out = (
23                config.output.output_dir
24                if not isinstance(config, dict)
25                else config.get("output", {}).get("output_dir")
26            )
27        except Exception:
28            out = None
29        return out or "output"
30
31
32     def safe_output_filename(name: str) -> str:
33         """Validate an output filename is a BARE name (no dir part, no traversal, not
34         absolute).
35         ``os.path.join(output_dir, name)`` silently discards ``output_dir`` when ``name`` is
36         absolute,
37         and ``..`` segments escape it ? so an unchecked ``output_filename`` is an arbitrary-
38         write
39         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
40         """
41         if not name or not name.strip():
42             raise ValueError("output_filename must not be empty")
43         if "\x00" in name:
44             raise ValueError("output_filename must not contain a null byte")
45         if "/" in name or "\\" in name:
46             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
47         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
48             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
49         return name
50

```

```

44
45     def resolve_under_root(path: str, root: str) -> str:
46         """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
47         rp = os.path.realpath(path)
48         rr = os.path.realpath(root)
49         try:
50             contained = os.path.commonpath([rp, rr]) == rr
51         except ValueError: # different drives on Windows
52             contained = False
53         if not contained:
54             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
55         return rp
56
57
58     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
59         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
60
61         With no allowed_root (default, single-user local mode) only the null-byte check
applies,
62         so the local 'point it at any of my videos' workflow is preserved. Setting an
63         allowed_root (a future hosted/tenant deployment) enforces containment.
64         """
65         if "\x00" in path:
66             raise ValueError("video_path must not contain a null byte")
67         if allowed_root:
68             resolve_under_root(path, allowed_root)
69

```

49. assistant (2026-07-02T04:52:44.196Z)

Now I have full context. Implementing ? first the shared timeout constants/helper in the ffmpeg resolver seam.

50. user (2026-07-02T04:52:57.130Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/utils/ffmpeg.py has been updated successfully. (file state is current in your context ? no need to Read it back)

51. user (2026-07-02T04:53:07.314Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/utils/ffmpeg.py has been updated successfully. (file state is current in your context ? no need to Read it back)

52. assistant (2026-07-02T04:53:11.818Z)

Now the compiler ? probe, per-clip encode, and concat.

53. user (2026-07-02T04:53:14.754Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/stitcher/compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

54. user (2026-07-02T04:53:25.064Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/stitcher/compiler.py has been updated successfully. (file state is

current in your context ? no need to Read it back)

55. assistant (2026-07-02T04:53:29.991Z)

Now the per-clip encode with the timeout-aborts-compile semantics.

56. user (2026-07-02T04:53:40.172Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/stitcher/compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

57. user (2026-07-02T04:53:51.169Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/stitcher/compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

58. user (2026-07-02T04:53:59.909Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/stitcher/compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

59. user (2026-07-02T04:54:09.649Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/stitcher/compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

60. assistant (2026-07-02T04:54:13.734Z)

Now the three validators.

61. user (2026-07-02T04:54:16.767Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/validators/format.py has been updated successfully. (file state is current in your context ? no need to Read it back)

62. user (2026-07-02T04:54:25.125Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/validators/format.py has been updated successfully. (file state is current in your context ? no need to Read it back)

63. user (2026-07-02T04:54:33.637Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/validators/format.py has been updated successfully. (file state is current in your context ? no need to Read it back)

64. user (2026-07-02T04:54:40.804Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-

timeouts/backend/pipeline/validators/pts_continuity.py has been updated successfully. (file state is current in your context ? no need to Read it back)

65. user (2026-07-02T04:54:43.883Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/validators/pts_continuity.py has been updated successfully. (file state is current in your context ? no need to Read it back)

66. user (2026-07-02T04:54:53.337Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/validators/pts_continuity.py has been updated successfully. (file state is current in your context ? no need to Read it back)

67. user (2026-07-02T04:55:06.907Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/validators/resolution_fps.py has been updated successfully. (file state is current in your context ? no need to Read it back)

68. assistant (2026-07-02T04:55:12.422Z)

Now `backend/utils/video.py` (probe + chunk extraction).

69. user (2026-07-02T04:55:20.063Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/utils/video.py has been updated successfully. (file state is current in your context ? no need to Read it back)

70. user (2026-07-02T04:55:29.041Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/utils/video.py has been updated successfully. (file state is current in your context ? no need to Read it back)

71. assistant (2026-07-02T04:55:35.964Z)

Now the gemini debug-trim call site.

72. user (2026-07-02T04:55:38.820Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/segmenters/gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

73. user (2026-07-02T04:55:42.901Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/segmenters/gemini.py has been updated successfully. (file state is current in your context ? no need to Read it back)

74. user (2026-07-02T04:55:51.103Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-

indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/utils/annotation_proxy.py has been updated successfully. (file state is current in your context ? no need to Read it back)

75. user (2026-07-02T04:55:54.723Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/utils/annotation_proxy.py has been updated successfully. (file state is current in your context ? no need to Read it back)

76. user (2026-07-02T04:56:03.577Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/utils/annotation_proxy.py has been updated successfully. (file state is current in your context ? no need to Read it back)

77. user (2026-07-02T04:56:11.103Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/audio/hit_detector.py has been updated successfully. (file state is current in your context ? no need to Read it back)

78. user (2026-07-02T04:56:14.864Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/pipeline/audio/hit_detector.py has been updated successfully. (file state is current in your context ? no need to Read it back)

79. assistant (2026-07-02T04:56:18.981Z)

Now update the 02 RISK comment in job_worker.py.

80. user (2026-07-02T04:56:26.390Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/backend/api/job_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

81. assistant (2026-07-02T04:56:35.690Z)

Now the tests. First extend `tests/test\_ffmpeg\_resolver.py` with the timeout-policy units.

82. user (2026-07-02T04:56:46.431Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/tests/test_ffmpeg_resolver.py has been updated successfully. (file state is current in your context ? no need to Read it back)

83. assistant (2026-07-02T04:56:56.018Z)

Now the compiler timeout tests.

84. user (2026-07-02T04:57:17.790Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/tests/test_compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

context ? no need to Read it back)

85. user (2026-07-02T04:57:36.738Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py:72:def acquire_local_input(C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-73- raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-74-) -> str:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-75- """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for processing.
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-76-
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-77- Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's behaviour
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-78- byte-for-byte). A bucket / Drive source (`gs://`/`s3://`/`gdrive://`, or a bare name under a
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-79- non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir and the local
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-80- path returned. The scratch copy is left under the system temp dir (a pure intermediate,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-81- regenerable from the source) and relies on OS temp reclamation ? the same pull pattern the
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-82- worker's ``cmd_infer`` already ships."""
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-83- sc = _storage_dict(storage_config)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-84- ref = resolve_input_ref(raw, sc)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-85- if ref.backend == "local":
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-86- return (
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-87- ref.key
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-timeouts\backend\storage\_init__.py-88- ) # identity ? no temp, no copy, byte-identical
to passing the path directly
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
```

```

timeouts\backend\storage\__init__.py-89-     scratch = tempfile.mkdtemp(prefix="rally-input-",
dir=scratch_parent)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\storage\__init__.py-90-     return fetch(ref.uri, os.path.join(scratch, ref.name
or "in.mp4"), config=sc)
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\storage\__init__.py-91-
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\storage\__init__.py-92-
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\storage\__init__.py-93-def _backend_class(name: str):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\storage\__init__.py-94-     """Lazy-resolve a backend class so cloud SDKs aren't
imported until needed."""
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\storage\__init__.py-95-     if name == "local":
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\storage\__init__.py-96-         from backend.storage.local import LocalStorage
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-ffmpeg-
timeouts\backend\storage\__init__.py-97-

```

86. assistant (2026-07-02T04:57:41.517Z)

Now the validator timeout tests (new file).

87. assistant (2026-07-02T04:57:52.127Z)

"""R2-4: the process-path ffprobe probes must be timeout-bounded and a hung probe must become a CLEAN validation failure (? the job fails with a reason), never a hang or a crash. ffprobe is mocked to raise TimeoutExpired ? no real media, fully offline."""

```

import subprocess
from unittest.mock import patch

from backend.config.models import AppConfig
from backend.pipeline.validators.format import FormatValidator
from backend.pipeline.validators.pts_continuity import PTSContinuityValidator
from backend.pipeline.validators.resolution_fps import (
    ResolutionFPSValidator,
    _probe_meta,
)
from backend.utils.ffmpeg import FFPROBE_TIMEOUT_SEC

def _timeout_run(cmd, *a, **k):
    raise subprocess.TimeoutExpired(cmd, k.get("timeout", FFPROBE_TIMEOUT_SEC))

def test_format_validator_probe_carries_timeout(tmp_path):
    video = tmp_path / "v.mp4"
    video.write_bytes(b"x")
    captured = {}

    def fake_run(cmd, *a, **k):
        captured["timeout"] = k.get("timeout")
        raise subprocess.CalledProcessError(1, cmd, stderr="bad")

```



```

with patch.object(subprocess, "run", side_effect=fake_run):
    FormatValidator().validate(str(video), AppConfig(), {})
assert captured["timeout"] == FFPROBE_TIMEOUT_SEC

def test_format_validator_timeout_is_clean_distinct_failure(tmp_path):
    video = tmp_path / "v.mp4"
    video.write_bytes(b"x")
    with patch.object(subprocess, "run", side_effect=_timeout_run):
        res = FormatValidator().validate(str(video), AppConfig(), {})
    assert res.passed is False
    assert "timed out" in res.message

def test_pts_validator_timeout_is_clean_distinct_failure(tmp_path):
    video = tmp_path / "v.mp4"
    video.write_bytes(b"x")
    with patch.object(subprocess, "run", side_effect=_timeout_run):
        res = PTSContinuityValidator().validate(str(video), AppConfig(), {})
    assert res.passed is False
    assert "timed out" in res.message

def test_resolution_fps_reprobe_timeout_degrades_to_clean_failure(tmp_path):
    """With no shared ffprobe_meta the validator re-probes; a hung re-probe must return
    None from _probe_meta (TimeoutExpired ? SubprocessError) ? a clean failed result."""
    video = tmp_path / "v.mp4"
    video.write_bytes(b"x")
    with patch.object(subprocess, "run", side_effect=_timeout_run):
        assert _probe_meta(str(video)) is None
        res = ResolutionFPSValidator().validate(str(video), AppConfig(), {})
    assert res.passed is False
    assert "metadata is missing" in res.message

```

88. user (2026-07-02T04:57:52.158Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/tests/test_validator_ffmpeg_timeouts.py (file state is current in your context ? no need to Read it back)

89. user (2026-07-02T04:58:10.107Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/tests/test_video.py has been updated successfully. (file state is current in your context ? no need to Read it back)

90. assistant (2026-07-02T04:58:17.897Z)

Now update the annotation-proxy stub (it must accept the new `timeout` kwarg) and add timeout tests.

91. user (2026-07-02T04:58:23.177Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-ffmpeg-timeouts/tests/test_annotation_proxy.py has been updated successfully. (file state is current in your context ? no need to Read it back)

92. assistant (2026-07-02T04:58:23.692Z)

You've hit your session limit · resets 1:10pm (Asia/Calcutta)